XVERA LABS SOFTWARE HOUSE

PYTHON DJANGO DRF FASTAPI & AI

Complete Interview Preparation Guide

Core Python | OOP | DSA | Django | DRF | FastAPI | AI/ML

425+ SAWAL O JAWAB

ROMAN URDU EDITION

MUHAMMAD NOUMAN

Founder & CEO, Xvera Labs

Python, Django, DRF, FastAPI & AI

PYTHON | DJANGO | DRF
FASTAPI | AI

Complete Interview Preparation Guide

425 Sawal o Jawab (Roman Urdu)

Muhammad Nouman
Founder & CEO, Xvera Labs

Copyright © 2026 Muhammad Nouman. Tamam huqooq mehfooz hain.
Shaya karda: Xvera Labs Software House.

Musannif ki likhi ijazat ke baghair is kitab ka koi hissa kisi bhi shakal me copy, store ya share nahi kiya ja sakta, siwaye review ke liye chhote iqtibas ke.

Ye kitab seekhne aur interview ki practice ke liye hai. Technology tezi se badalti hai, is liye taza tareen tafseel ke liye official documentation zaroor dekhein.

Pehla edition.

Musannif ki taraf se

Assalam o Alaikum aur khush amdeed. Mera naam Muhammad Nouman hai. Main Python backend engineer hoon aur Xvera Labs chalata hoon, jahan hum poori duniya ke clients ke liye software banate hain. Main roz Python, Django, DRF, PostgreSQL, Redis, Celery aur Docker par kaam karta hoon.

In saalon me maine bohat se qabil developers ko sirf ek wajah se job offer kho dete dekha hai. Unhe kaam aata tha, lekin wo interview me saaf tareeqe se samjha nahi paate the. Ye kitab isi masle ko theek karne ke liye banayi gayi hai.

Yahan har sawal wo hai jo asli interviewer poochte hain. Har jawab chhote aur aasaan alfaaz me likha gaya hai. Maine mushkil lafz jaan boojh kar nahi use kiye. Agar aap baat samajh gaye, to aap use apne alfaaz me keh sakte hain, aur interviewer yahi sunna chahta hai.

Ek waqt me ek chapter parhein. Jawab ko dhaank dein, zabaan se bol kar batayein, phir khud ko check karein. Ye kuch baar karenge to baatein yaad reh jayengi. Dua hai ke ye kitab aapko wo job dilane me madad kare jis ke aap haqdar hain.

Muhammad Nouman
Founder & CEO, Xvera Labs

Is Kitab Ko Kaise Use Karein

1. Chapter ba chapter chalein. Chapters basic se advanced ki taraf jate hain, is liye har chapter agle ki tayyari karta hai.

2. Khud ko test karein. Sawal parhein, jawab dhaank dein aur zabaan se bolne ki koshish karein. Phir jawab parh kar kami door karein.

3. Apne alfaaz use karein. Rattaa na lagayein. Baat ko samjhein, phir usay apne andaaz me bayan karein.

4. Code chala kar dekhein. Jahan code example ho, usay khud type karke chalayein. Chalta dekhne se baat yaad reh jati hai.

5. Kamzor hisson ko dobara dekhein. Jo sawal chhoot jayein unhe mark karein aur interview se pehle dobara parhein.

Fehrist (Contents)

Chapter 1: Python Core & Advanced Concepts 9

Chapter 2: Object-Oriented Programming (OOP) 28

Chapter 3: Data Structures & Algorithms (DSA) 38

Chapter 4: Django Framework 50

Chapter 5: Django REST Framework (DRF) 61

Chapter 6: FastAPI Framework 68

Chapter 7: Artificial Intelligence & Machine Learning 77

Sawalon ka Nakshah

Bab	Topic	Sawal
1	Python Core & Advanced Concepts	Q1 - Q91 (91)
2	Object-Oriented Programming (OOP)	Q92 - Q138 (47)
3	Data Structures & Algorithms (DSA)	Q139 - Q205 (67)
4	Django Framework	Q206 - Q273 (68)
5	Django REST Framework (DRF)	Q274 - Q320 (47)
6	FastAPI Framework	Q321 - Q370 (50)
7	Artificial Intelligence & Machine Learning	Q371 - Q425 (55)

CHAPTER 1

Python Core & Advanced Concepts

1

91 sawal: Python ki mazboot buniyad

Is chapter me Python ki asal buniyad hai: data types, functions, scope, generators, decorators, threads, memory aur bohat kuch. Interview hamesha yahin se shuru hota hai, is liye har jawab ko apne alfaaz me samjha kar bolna seekhein.

Q1. What is Python?

Python ek high-level, interpreted, dynamically typed aur object-oriented programming language hai. Ye simple syntax ki wajah se famous hai aur web development, AI, automation, scripting aur data science me use hoti hai.

Features: easy aur readable syntax, bara community, cross-platform, huge libraries, aur variable ka type pehle se likhna nahi parta.

Example: print("Hello World")

Q2. Python interpreted language q kehlati hai?

Python ko interpreted language is liye kehte hain ke hume code ko pehle alag se compile karke executable file banane ki zarurat nahi parti. Interpreter source code ko khud bytecode me badal kar Python Virtual Machine (PVM) se chala deta hai.

Dhyan rahe: PVM bytecode instructions chalati hai, ye source ki lines ko seedha 'line by line' nahi parhti.

Q3. Python interpreted hai ya compiled?

Python dono tareeqe use karti hai. Pehle step me source code bytecode me compile hota hai (.pyc file, __pycache__ folder me), phir PVM us bytecode ko execute karti hai. Isi liye usay aam tor par interpreted language kaha jata hai.

Asal me compiled ya interpreted hona language ki nahi, uski implementation ki baat hai. CPython bytecode interpret karta hai, PyPy JIT compiler use karta hai.

Q4. What are Data Types in Python?

Data type batata hai ke variable me kis qism ka data hai aur us par kaun se operations ho sakte hain. Python me type variable ka nahi balke object ka hota hai.

Built-in types: Numeric (int, float, complex), Text (str), Sequence (list, tuple, range), Mapping (dict), Set (set, frozenset), Boolean (bool), Binary (bytes, bytearray, memoryview), aur None (NoneType).

```
type(10)      # int
type([1, 2])  # list
type(None)    # NoneType
```

Q5. Variables kia hote hain?

Variable ek naam (label) hai jo kisi object ko refer karta hai. Python me variable koi dabba nahi hota jisme value rakhi jaye; a = 10 ka matlab hai ke naam 'a' int object 10 ko point kar raha hai.

Isi liye ek hi object ko kai naam diye ja sakte hain, aur variable ka type fix nahi hota (dynamic typing).

```
a = 10
a = "hello"   # ab a string object ko point karta hai
```

Q6. What is mutable and immutable in python?

Mutable objects wo hain jinhein banane ke baad usi object me badla ja sakta hai. Example: list, dict, set, bytearray.

Immutable objects wo hain jin ki value ek baar ban jane ke baad badli nahi ja sakti. Example: int, float, bool, str, tuple, frozenset, bytes.

Nuqta: tuple khud immutable hai lekin agar uske andar list ho to wo list badli ja sakti hai: t = (1, [2]); t[1].append(3) chal jata hai. Immutable objects hashable hote hain, isliye dict key ban sakte hain.

Q7. Difference between List and Tuple?

List mutable hoti hai, square brackets [ ] me likhi jati hai, aur uske methods zyada hain (append, remove, sort). Tuple immutable hoti hai, parentheses ( ) me likhi jati hai, kam memory leti hai aur list se thori tez hoti hai.

Tuple hashable hoti hai (agar andar ke elements hashable hon) isliye dict key ban sakti hai, list nahi bana sakti. Fixed data (coordinates, function se multiple values return) ke liye tuple, badalne wale data ke liye list use karein.

```
t = (1,)    # ek element ka tuple, comma zaroori hai
x = (1)     # ye int hai, tuple nahi
```

Q8. What is Type Casting in Python?

Aik data type ko dusre data type me convert karna Type Casting kehlata hai. Do types hoti hain: Implicit - Python automatically convert karta hai. Explicit - Hum manually conversion karte hain.

Q9. Python me kitni type ke Operators hote hain?

Python me 7 tarah ke operators hain:

1. Arithmetic: + - * / // % ** (/ hamesha float deta hai, // floor division hai)

2. Comparison: == != > < >= <=

3. Logical: and, or, not

4. Assignment: = += -= *= /= waghera

5. Membership: in, not in

6. Identity: is, is not

7. Bitwise: & | ^ ~ << >>

```
7 / 2    # 3.5
7 // 2   # 3
7 % 2    # 1
2 ** 3   # 8
```

Q10. What is a function?

Function aik reusable block of code hota hai jo specific task perform karta hai.

```
def greet(name):
    return f"hello {name}"
```

Q11. Difference between parameter and argument?

Parameter wo variable hota hai jo function define karte waqt parentheses ke andar likha jata hai. Argument wo actual value hoti hai jo function call karte waqat pass ki jati hai.

Q12. Types of Arguments in Python?

Python me function arguments ki ye types hain:

1. Positional: order ke mutabiq pass hote hain. 2. Keyword: naam ke sath pass hote hain (order matter nahi). 3. Default: parameter ki pehle se value set hoti hai. 4. Variable-length: *args (tuple) aur **kwargs (dict). 5. Keyword-only: * ke baad wale parameters, sirf naam ke sath pass ho sakte hain. 6. Positional-only: / se pehle wale parameters (Python 3.8+).

```
def f(a, b=2, *args, c, **kwargs): ...
# c keyword-only hai: f(1, c=5)
```

Q13. Positional arguments kia hote hain?

Jaha order (tarteeb) matter karti hai.

```
def student(name, math, science):
    print(f"{name} got {math}, {science}")
student("Ali", 80, 90)
```

Q14. Keyword arguments kia hote hain?

Function call karte waqat parameter ka naam likh kar value pass ki jati hai.

Example: student(name="Ali", age=22)

Q15. Default arguments kia hote hain?

Jaha parameter ki value pehle se set hoti hai.

```
def greet(name, message="Hello"):
    print(f"{message}, {name}")
```

Q16. *args kya hai?

*args multiple positional arguments ko ek tuple ki tarah store karta hai, jab hume nahi pata hota ke function me kitni values ayengi.

Q17. **kwargs kya hai?

**kwargs multiple keyword arguments ko ek dictionary ki tarah receive karta hai.

Q18. What is the lambda function?

Lambda function aik anonymous (bina naam ka) function hota hai jo sirf ek expression rakhta hai.

Example: square = lambda x: x**2

Q19. What is map()?

map() kisi function ko iterable ke har element par lagata hai. Python 3 me ye list nahi balke lazy iterator return karta hai, isliye list() se list banani parti hai.

```
list(map(lambda x: x * 2, [1, 2, 3]))   # [2, 4, 6]
```

Q20. What is filter()?

filter() sirf un elements ko return karta hai jo condition ko satisfy karte hain.

Example: list(filter(lambda x: x%2==0, [1,2,3,4])) -> [2,4]

Q21. What is reduce()?

reduce() iterable ki tamam values ko combine karke ek single value return karta hai.

Example: from functools import reduce; reduce(lambda x,y: x*y, [1,2,3,4]) -> 24

Q22. What is scope in Python?

Scope batata hai ke variable program mein kahan se accessible hai.

Python variable ko **LEGB** order mein dhoondta hai:

- **L - Local:** Current function ke andar.
- **E - Enclosing:** Outer function ke andar.
- **G - Global:** Function ke bahar, module level par.
- **B - Built-in:** Python ke built-in names, jaise `len()` aur `print()`.

`global` keyword global variable ki value change karne ke liye use hota hai, jabke `nonlocal` enclosing function ke variable ki value change karne ke liye use hota hai.

Q23. UnboundLocalError kab aata hai?

Jab function ke andar kisi local variable ko value dene se pehle uski value read karne ki koshish karein, to `UnboundLocalError` aata hai.

Agar global variable ko function ke andar change karna ho, to `global` keyword use karte hain.

Example:

```
count = 0

def inc():
    count = count + 1  # UnboundLocalError

def inc_ok():
    global count
    count = count + 1  # Theek hai
```

Yaad rakhne ki trick:

- Sirf global variable read karna ho → `global` ki zaroorat nahi.
- Global variable change karna ho → `global` keyword use karein.
- Local variable ko value dene se pehle read karein → `UnboundLocalError` aa sakta hai.

Q24. What is recursion?

Recursion aik technique hai jisme function khud ko call karta hai.

Ismein 2 cheezein hoti hain:

1. Base case: Jahan function ruk jata hai.
2. Recursive case: Jahan function khud ko dobara call karta hai.

Example: `factorial(n) = n * factorial(n-1)`

Q25. What is exception handling?

Exception handling runtime errors ko pakar kar program ko crash hone se bachati hai.

try: wo code jisme error aa sakta hai. except: error aane par chalta hai. else: sirf tab chalta hai jab try me koi error na aaye. finally: hamesha chalta hai (cleanup ke liye). raise: khud exception paida karna.

```
try:
    r = 10 / int(input())
except ZeroDivisionError:
    print("zero se divide nahi")
except ValueError:
    print("number daalein")
else:
    print(r)
finally:
    print("hamesha chalega")
```

Q26. Difference between == and is?

== do objects ki value compare karta hai. is do objects ki memory location compare karta hai.

Q27. Deep copy vs shallow copy?

Deep copy poore object aur nested objects ki alag copy banati hai (original change ka koi asar nahi). Shallow copy naya object banati hai lekin nested objects ka reference share karti hai.

Q28. Iterable vs Iterator?

Iterable: Aisa object jis par `for` loop chala sakte hain, jaise list, tuple aur string.

Iterator: Aisa object jo `next()` se ek ek value deta hai. Values khatam hone par `StopIteration` deta hai.

Yaad rakhein: Har iterator iterable hota hai, lekin har iterable iterator nahi hota.

Q29. What is a generator?

Generator aik special function hota hai jo yield keyword use karta hai aur values ko one by one generate karta hai, sari values ek sath memory me store nahi karta.

Q30. yield aur return me kya difference hai?

'return' function ko completely khatam kar deta hai. 'yield' function ko pause karta hai aur agli call par wahin se continue karta hai.

Q31. Generator kab use karte hain?

Large files read karne ke liye, large datasets process karne ke liye, memory optimization ke liye, streaming data ke liye.

Q32. Generator dobara use kyun nahi hota?

Generator ek one-time iterator hota hai. Values consume hone ke baad dobara use nahi hota, naya generator banana parta hai.

Q33. What is a decorator?

Decorator aik function hai jo kisi dusre function ki functionality ko bina change kiye modify ya extend karta hai. @ syntax se use hota hai. Common uses: Logging, Authentication, Timing, Caching, Validation.

Q34. Kitni types ke decorators hote hain?

Function decorators, Class decorators, aur Built-in decorators (@staticmethod, @classmethod, @property).

Q35. @property decorator kya karta hai?

@property kisi method ko attribute ki tarah access karne deta hai, bina parentheses ke call kiye.

Q36. What is Middleware in general terms?

Middleware wo component hota hai jo request aur response ke darmiyan kaam karta hai aur unhein process karta hai.

Q37. What is Synchronous Programming?

Synchronous Programming me code line by line execute hota hai. Ek task complete hone ke baad hi dusra task start hota hai.

Q38. What is Asynchronous Programming?

Asynchronous Programming me jab ek task wait (I/O operation) me ho to program dusra task start kar sakta hai. Ye application ko zyada efficient banata hai (event loop ke through concurrently).

Q39. What are async and await?

- `async`: Function ko asynchronous function banane ke liye use hota hai.
- `await`: Kisi asynchronous task ke complete hone ka intezar karta hai aur is dauran doosre tasks ko chalne ka mauqa deta hai.

In dono ki madad se program waiting time ko efficiently use kar sakta hai.

```
import asyncio

async def work():
    await asyncio.sleep(1)
    print("Work complete")

asyncio.run(work())
```

Yaad rakhne ki trick:

- `async` = Asynchronous function.
- `await` = Asynchronous task ka intezar, bina event loop ko block kiye.

Q40. What is GIL in Python?

GIL (Global Interpreter Lock) CPython ka ek lock hai jo ek waqt mein sirf ek thread ko Python bytecode execute karne deta hai.

- CPU-bound tasks: Multithreading se aam tor par speed nahi barhti.
- I/O-bound tasks: Multithreading faida deti hai kyun ke I/O wait ke dauran doosre threads kaam kar sakte hain.
- Multiprocessing: CPU-bound tasks ko multiple processes mein chalane ke liye use hoti hai.

Q41. What is Multithreading?

Multithreading mein ek process ke andar multiple threads hoti hain jo memory share karti hain. Ye I/O-bound tasks ke liye useful hai, jaise API calls aur file operations.

Q42. What is Multiprocessing?

Multiprocessing mein multiple processes hoti hain, aur har process ki apni memory hoti hai. Ye CPU-bound tasks ke liye useful hai, jaise heavy calculations.

Q43. Multithreading aur Multiprocessing mein kya difference hai?

Multithreading ek process mein multiple threads chalati hai aur memory share karti hai. Multiprocessing multiple processes chalati hai, jin ki memory alag hoti hai. Multithreading I/O-bound aur multiprocessing CPU-bound tasks ke liye useful hai.

Q44. Agar GIL ek waqt mein ek thread chalne deta hai, to multithreading ka kya faida?

I/O-bound tasks mein jab ek thread data ya response ka wait karti hai, to doosri thread kaam kar sakti hai. Is liye multithreading useful hai.

Q45. Kya GIL multiprocessing ko affect karta hai?

Nahi, aam CPython multiprocessing mein har process ka apna GIL hota hai, is liye multiple processes CPU par parallel kaam kar sakti hain.

Q46. What is a CPU-bound task?

Aisa task jisme zyada waqt CPU calculations karne mein lagta hai, jaise heavy mathematical calculations. Iske liye multiprocessing useful ho sakti hai.

Q47. What is an I/O-bound task?

Aisa task jisme zyada waqt data read/write ya network response ka wait karne mein lagta hai, jaise API call ya file reading. Iske liye multithreading useful ho sakti hai.

Q48. What is Garbage Collection?

Garbage Collection aik automatic memory management process hai jo un objects ki memory free karta hai jo program me ab use nahi ho rahe.

Q49. Python mein Garbage Collection kaise kaam karti hai?

Python mein Garbage Collection memory clean karne ke liye use hoti hai. Ye do tareeqon se kaam karti hai:

1. Reference Counting: Jab object ke references zero ho jate hain, to uski memory release ho sakti hai.
2. Cyclic Garbage Collection: Aise objects ko detect karti hai jo circular references ki wajah se reference counting se clean nahi ho pate.

Q50. Python memory management ke liye kya use karta hai?

Reference Counting aur Garbage Collection.

Q51. What is a module in Python?

Module ek .py file hoti hai jisme functions, classes ya variables define hote hain aur usay import karke dusri files me use kia ja sakta hai.

Q52. What is a package in Python?

Package modules ka folder hota hai. Traditional tareeqe me us folder me __init__.py file hoti hai jo usay package banati hai. Python 3.3+ me __init__.py ke baghair bhi folder 'namespace package' ki tarah import ho sakta hai, lekin aam projects me __init__.py rakhna behtar hai.

```
from mypkg.utils import helper
```

Q53. Difference between module and package?

Module ek single file hoti hai, jabke package multiple modules ka folder hota hai.

Q54. What are Python virtual environments?

Virtual environment ek isolated environment hota hai jisme project-specific packages install kiye jate hain taake different projects ke dependencies conflict na karein.

Q55. pip kya hai?

pip Python ka package manager hai jo libraries install, upgrade aur remove karne ke liye use hota hai.

Q56. What is PEP 8?

PEP 8 Python ki official style guide hai jo code ko readable aur consistent banane ke liye rules deti hai.

Q57. What is a docstring?

Docstring triple quotes me likha gaya documentation string hota hai jo function/class/module ka purpose describe karta hai.

Q58. List comprehension kya hai?

List comprehension ek concise tarika hai list banane ka.

Example: squares = [x**2 for x in range(5)]

Q59. Dictionary comprehension kya hai?

Dictionary comprehension ek concise tarika hai dictionary banane ka.

Example: {x: x**2 for x in range(5)}

Q60. What is slicing in Python?

Slicing sequence (list, string, tuple) ka ek portion nikalne ke liye use hota hai.

Example: a[1:4], a[::-1] (reverse)

Q61. Difference between append() and extend()?

append() ek single element list ke end me add karta hai. extend() ek iterable ke saare elements list me add karta hai.

Q62. Difference between remove(), pop() and del?

remove(value): pehli matching value hatata hai (na mile to ValueError), kuch return nahi karta. pop(index): index par se element hata kar return karta hai (default aakhri element). del a[i]: index ya slice ya poora variable delete karta hai, kuch return nahi karta. clear(): list khali kar deta hai.

```
a = [10, 20, 30, 20]
a.remove(20)   # [10, 30, 20]
x = a.pop()    # x = 20, a = [10, 30]
del a[0]       # a = [30]
```

Q63. What is a set in Python?

Set unordered collection hoti hai jisme duplicate values allowed nahi hoti.

Q64. Difference between set and frozenset?

Set mutable hota hai, frozenset immutable hota hai.

Q65. What are Python namespaces?

Namespace ek naming system hai jo har identifier (variable, function, class) ko unique naam deta hai taake naam clash na ho.

Q66. Containers (list, dict) par == aur is ka farq kya hai?

Containers (list, dict) par == un ke elements ko aapas me barabar check karta hai (andar ke nested elements bhi compare hote hain). 'is' sirf ye dekhta hai ke dono naam ek hi object ko point kar rahe hain ya nahi (identity).

```
a = [1, [2, 3]]
b = [1, [2, 3]]
a == b   # True, values barabar hain
a is b   # False, alag objects hain
```

Q67. What are Python data classes?

@dataclass decorator classes banane ka boilerplate kam karta hai, jaise __init__, __repr__ automatically generate ho jate hain.

Q68. What is context manager (with statement)?

Context manager resources (files, connections) ko automatically open aur close karne ke liye use hota hai.

Example: with open('file.txt') as f: data = f.read()

Q69. __enter__ aur __exit__ methods kya karte hain?

Ye custom context manager banane ke liye use hote hain. __enter__ resource setup karta hai, __exit__ cleanup karta hai.

Q70. What are magic/dunder methods?

Magic methods Python ke special methods hote hain jin ke naam double underscores (__) se start aur end hote hain, jaise __init__, __str__, __len__, __add__.

Q71. __str__ vs __repr__?

__str__ user-friendly readable string return karta hai (print ke liye). __repr__ developer-friendly unambiguous string return karta hai (debugging ke liye).

Q72. What is duck typing in Python?

Duck typing me object ka type important nahi hota, balke uske pass required method/attribute hona chahiye. "If it walks like a duck and quacks like a duck, it's a duck."

Q73. What is monkey patching?

Monkey patching runtime par kisi class ya module ke behavior ko dynamically change karna hota hai.

Q74. Normal function aur closure me kya farq hai?

Normal function apne local variables sirf apni call ke dauran yaad rakhta hai. Closure wo inner function hai jo apne enclosing (outer) function ke variables ko yaad rakhta hai, chahe outer function khatam ho chuka ho.

```
def make_adder(n):
    def add(x):
        return x + n   # n outer function se yaad rakha
    return add
add5 = make_adder(5)
add5(10)   # 15
```

Q75. What is a closure in Python?

Closure ek inner function hota hai jo outer function ki variables ko access kar sakta hai, chahe outer function ka execution khatam ho chuka ho.

```
def outer(x):
    def inner():
        return x
    return inner
```

Q76. What is the difference between shallow copy of list using slicing and copy()?

Dono shallow copy hi karte hain lekin nested objects ka reference same rehta hai; dono me farq sirf syntax ka hai.

Q77. What is pickling and unpickling?

Pickling Python object ko byte stream me convert karna hai (serialization). Unpickling byte stream ko wapas Python object me convert karna hai (deserialization).

Q78. What is the difference between json and pickle?

JSON human-readable text format hai, language-independent hai (APIs me use hota hai) aur sirf basic types (dict, list, str, number, bool, null) support karta hai. Pickle Python-specific binary format hai jo zyada tar Python objects (custom classes bhi) serialize kar leta hai.

Security: untrusted source ka pickle data kabhi load na karein, kyun ke unpickle karte waqt arbitrary code chal sakta hai.

```
import json, pickle
json.dumps({'a': 1})   # '{"a": 1}'
pickle.dumps([1, 2])   # bytes
```

Q79. What is the walrus operator (:=)?

Walrus operator ek expression ke andar variable assign karne deta hai.

Example: if (n := len(data)) > 10: print(n)

Q80. What are type hints in Python?

Type hints function parameters, return values aur variables ke expected types batate hain (documentation aur tools ke liye). Python runtime par inhein enforce nahi karti; checking mypy jaise tools ya IDE karte hain. FastAPI aur Pydantic type hints ko asli validation ke liye use karte hain.

```
def add(a: int, b: int) -> int:
    return a + b
add('x', 'y')   # runtime par chal jata hai, error nahi
```

Q81. LEGB rule kya hai?

Python variable ka naam LEGB order me dhoondta hai: Local (function ke andar), Enclosing (bahar wale function me), Global (module level), Built-in (len, print jaise). Jahan pehle mil jaye wahin ruk jata hai.

```
x = 'global'
def outer():
    x = 'enclosing'
    def inner():
        x = 'local'
        print(x)   # local
    inner()
```

Q82. What is the difference between is and == for immutable objects like small integers?

CPython -5 se 256 tak ke chhote integers pehle se cache karta hai, isliye a = 100; b = 100 par 'a is b' True aata hai. Bare numbers par (1000 waghera) alag objects ban sakte hain aur 'is' False de sakta hai.

Ye implementation detail hai, is par kabhi depend na karein. Values compare karne ke liye hamesha == use karein, aur 'is' sirf None check ke liye: if x is None.

Q83. What is the difference between shallow copy and assignment (=)?

Assignment (=) sirf reference copy karta hai, dono variables same object point karte hain. Shallow copy naya object banata hai lekin nested references share karta hai.

Q84. What is unpacking in Python?

Unpacking se list/tuple ki values alag variables me assign ki ja sakti hain.

Example: a, b, c = [1, 2, 3]

Q85. What is the zip() function?

zip() multiple iterables ko combine karke tuples ka ek iterator return karta hai.

Example: list(zip([1,2],[3,4])) -> [(1,3),(2,4)]

Q86. What is the enumerate() function?

enumerate() iterable ke elements ke sath unka index bhi return karta hai.

Example: for i, val in enumerate(['a','b']): print(i, val)

Q87. What is the difference between range() and list?

range() ek immutable sequence object hai jo values memory me store nahi karta balke zarurat par calculate karta hai, isliye bohat bari range bhi kam memory leti hai. Is me len(), indexing, slicing aur 'in' check chalte hain. List saari values memory me rakhti hai aur mutable hoti hai.

Dhyan: range generator nahi hai; generator ek baar chalta hai jabke range ko baar baar iterate kiya ja sakta hai.

```
r = range(1, 1000000)
len(r)       # 999999
r[10]        # 11
500 in r     # True
```

Q88. What is string formatting in Python (f-strings)?

f-strings variables ko directly string ke andar embed karne deta hai.

Example: name = 'Ali'; print(f'Hello {name}')

Q89. What is exception chaining (raise from)?

raise ... from ... ek exception ko dusri exception ke context ke sath raise karta hai taake original error ka pata chal sake.

Q90. Python me Exception hierarchy kya hai? (BaseException, Exception aur Error)

Python me tamam exceptions BaseException ki subclasses hain. Exception wo class hai jis se normal runtime errors (ValueError, TypeError, KeyError, ZeroDivisionError, OSError waghera) nikalte hain, aur aam tor par 'except Exception' inhein pakarta hai.

SystemExit, KeyboardInterrupt aur GeneratorExit direct BaseException se aate hain, isliye 'except Exception' inhein nahi pakarta. Python me 'Error' koi alag category nahi; SyntaxError, MemoryError, RecursionError waghera bhi Exception ki hi subclasses hain (SyntaxError code run hone se pehle aata hai, isliye usi file ke try/except se nahi pakra jata).

```
try:
    int('abc')
except Exception as e:
    print(type(e).__name__)   # ValueError
```

Q91. What is a custom exception?

Custom exception Exception class ko inherit karke banai jati hai taake specific error cases handle kiye ja sakein.

Example: class MyError(Exception): pass OOP asal software ko organize karne ka tareeqa hai. Is chapter me classes, objects, inheritance, polymorphism, encapsulation aur abstraction hain. Har concept ko chhoti si example ke sath samjhane ki practice karein.

CHAPTER 2

Object-Oriented Programming (OOP)

47 sawal: classes, objects aur OOP ke chaar pillars

2

OOP asal software ko organize karne ka tareeqa hai. Is chapter me classes, objects, inheritance, polymorphism, encapsulation aur abstraction hain. Har concept ko chhoti si example ke sath samjhane ki practice karein.

Q92. What is OOP?

OOP (Object-Oriented Programming) aik programming paradigm hai jisme hum real-world entities ko Classes aur Objects ki form me represent karte hain.

Q93. Class vs Object?

Class aik blueprint (naqsha) hoti hai jiski basis par objects banaye jate hain. Object class ka real instance hota hai jo memory me create hota hai.

Q94. Instance aur Object me difference?

Python me har cheez object hai (int, function, class bhi). Instance kisi khaas class ka object hota hai: d = Dog() me d, Dog class ka instance hai. Practically dono lafz aksar ek hi matlab me use hote hain, lekin 'instance of X' kehne se class batayi jati hai.

```
d = Dog()
isinstance(d, Dog)   # True
```

Q95. Kya ek class se multiple objects bana sakte hain?

Haan, ek class se multiple objects ya instances create kiye ja sakte hain.

Q96. Attributes kitni types ke hote hain?

2 types: Class Attributes (class ke andar, __init__ ke bahar define hote hain) aur Instance Attributes (__init__ ke andar self ke sath define hote hain).

Q97. Python me methods ki types kitni hoti hain?

3 types: Instance method (self), Class method (@classmethod, cls), Static method (@staticmethod).

Q98. Instance method kya hai?

Instance method self parameter leta hai aur object (instance) ke data ke sath kaam karta hai.

Q99. Class method kya hai?

@classmethod decorator wala method pehla parameter cls (class khud) leta hai aur class level ke data par kaam karta hai. Aam istemal: alternative constructor banana.

```
class Date:
    def __init__(self, d, m, y):
        self.d, self.m, self.y = d, m, y
    @classmethod
    def from_string(cls, s):
        d, m, y = map(int, s.split('-'))
        return cls(d, m, y)
Date.from_string('10-05-2024')
```

Q100. Static method kya hai?

@staticmethod decorator use karta hai. Ye na object ke data se related hota hai na class ke data se.

Q101. Self kya hota hai?

self current object (instance) ka reference hota hai.

Q102. What is constructor __init__?

Constructor __init__() Python ka special method hai jo object create hote hi automatically call hota hai. __new__() object create karta hai aur __init__() usay initialize karta hai.

Q103. What are Access Modifiers?

Access modifiers batate hain ke class ka data kaun use kare. Python me 3 conventions hain: Public (name), Protected (_name), Private (__name).

Dhyan: Java/C++ ki tarah Python inhein strictly enforce nahi karta. Public aur protected me sirf naming convention hai; private me name mangling hoti hai jis se bahar se direct access mushkil (lekin namumkin nahi) ho jata hai.

Q104. Public access modifier kya hai?

Har jagah se access kiya ja sakta hai. Bina underscore ke likha jata hai, jaise self.name.

Q105. Protected access modifier kya hai?

Protected attribute single underscore se likha jata hai (self._name). Ye ek convention hai ke is ko sirf class aur uski subclasses use karein. Python bahar se access ko rokta nahi, sirf developers ko ishara deta hai.

Q106. Private access modifier kya hai?

Private attribute double underscore se likha jata hai (self.__name). Python usay internally _ClassName__name me badal deta hai (name mangling), is liye bahar se obj.__name likhne par AttributeError aata hai. Ye security nahi, sirf accidental access aur subclass me naam takrao se bachane ke liye hai.

```
class A:
    def __init__(self):
        self.__x = 1
a = A()
# a.__x            -> AttributeError
a._A__x              # 1, abhi bhi access ho jata hai
```

Q107. Name mangling kya hai?

Jab private attribute (__name) define kiya jata hai to Python usay internally _ClassName__name me convert kar deta hai taake accidental overriding na ho.

Q108. Association kia hai?

Jab aik class ka object doosri class ke object se related ho, to usay Association kehte hain.

Q109. Kya Association me dono objects independently exist kar sakte hain?

Haan, dono objects ek dusre ke bina bhi exist kar sakte hain.

Q110. Aggregation kia hai?

Aggregation Association ki special type hai jahan ek class doosri class ka object rakhti hai lekin dono independently exist kar sakte hain (HAS-A relationship).

Q111. Aggregation aur Association me difference?

Association sirf relationship batati hai. Aggregation me HAS-A relationship hoti hai aur ek class doosri class ka object rakhti hai.

Q112. Aggregation vs Composition?

Aggregation weak HAS-A relationship hai jahan child object parent ke baghair exist kar sakti hai. Composition strong HAS-A relationship hai jahan child object parent ke bina exist nahi kar sakta.

Q113. Four Pillars of OOP kya hain?

Encapsulation, Inheritance, Polymorphism, Abstraction.

Q114. What is Encapsulation?

Encapsulation me data aur methods ko ek class ke andar bundle kiya jata hai aur data ko direct access se protect kiya jata hai.

Q115. Data aur methods ko secure kyun kiya jata hai?

Taake unauthorized access na ho aur data safe rahe.

Q116. Getter aur Setter kya hain?

Getter value parhne ka aur setter value set karne ka tareeqa hai, jis se validation lagayi ja sakti hai. Python me Java ki tarah getX/setX likhne ke bajaye @property use hota hai: pehle simple attribute rakhein, zarurat par property bana dein.

```
class Person:
    def __init__(self, age):
        self._age = age
    @property
    def age(self):
        return self._age
    @age.setter
    def age(self, v):
        if v < 0:
            raise ValueError('age negative nahi ho sakti')
        self._age = v
p = Person(20)
p.age = 25   # setter chala
```

Q117. What is Inheritance?

Inheritance me child class parent class ke methods ya properties ko inherit karti hai.

Q118. Types of Inheritance?

Single, Multi-level, Multiple, Hierarchical Inheritance.

Q119. Single Inheritance kya hai?

Jab ek child class ek parent class se inherit kare.

```
class Animal:
    def eat(self):
        print('Eating')
class Dog(Animal):
    pass
```

Q120. Multiple Inheritance kya hai?

Jab ek child class multiple parent classes se inherit kare.

Example: class Child(Father, Mother): pass

Q121. Multilevel Inheritance kya hai?

Jab child class parent class se aur parent class grandparent class se inherit kare.

Q122. Hierarchical Inheritance kya hai?

Jab multiple child classes ek hi parent class se inherit karein.

Q123. What is Polymorphism?

Polymorphism same method ya function different objects ke liye different behaviors provide karta hai.

Q124. Types of Polymorphism?

Polymorphism ka matlab hai ek hi naam ka method ya operator alag objects ke liye alag behavior de. Python me ye in tareeqon se milti hai: Method Overriding (child class parent ka method dobara likhe), Duck typing (jis object me method ho wo chal jata hai, type zaroori nahi), aur Operator overloading (dunder methods jaise __add__).

Method overloading Python me directly support nahi hoti (default arguments, *args ya functools.singledispatch se milta julta kaam hota hai). Isliye 'Overloading aur Overriding' ko Python ki polymorphism ki types kehna adhoora jawab hai.

Q125. What is Method Overloading?

Overloading ka matlab same naam ke methods alag parameters ke sath. Python me ye directly support nahi: agar same naam ke do methods likhein to aakhri wala pehle ko replace kar deta hai. Isko default arguments, *args/**kwargs ya functools.singledispatch se emulate kiya jata hai.

```
def area(a, b=None):
    return a * a if b is None else a * b
area(5)      # square
area(5, 3)   # rectangle
```

Q126. What is Method Overriding?

Child class parent class ke method ko same name se dobara define karti hai aur apna behavior implement karti hai.

Q127. What is Abstraction?

Sirf important cheezein dikhana aur implementation details hide karna Abstraction kehlata hai.

Q128. What is an abstract class in Python?

Abstract class ek aisi class hai jise directly instantiate nahi kia ja sakta, ismein abstract methods hote hain jo child class ko implement karne padte hain. ABC module se banayi jati hai.

Q129. What is an Interface concept in Python?

Python me formal interface nahi hota, lekin abstract classes ya protocols se interface jaisa behavior implement kia jata hai.

Q130. super() kia hai?

super() MRO (Method Resolution Order) me agli class ka proxy deta hai, taake child class se parent ka method ya constructor call kiya ja sake. Single inheritance me ye parent hota hai; multiple inheritance me ye 'direct parent' nahi balke MRO ki agli class hoti hai.

```
class Person:
    def __init__(self, name):
        self.name = name
class Employee(Person):
    def __init__(self, name, salary):
        super().__init__(name)
        self.salary = salary
```

Q131. Diamond problem kia hai?

Diamond problem multiple inheritance me tab aati hai jab class D do classes (B, C) se inherit kare aur B, C dono common parent A se inherit karti hon. Sawal ye banta hai ke D me A ka method kis raste se aaye.

Python isay MRO (C3 linearization) se hal karta hai: ek fixed tarteeb tay ho jati hai aur har class sirf ek baar aati hai. D.__mro__ se wo

tarteeb dekhi ja sakti hai.

```
class A: pass
class B(A): pass
class C(A): pass
class D(B, C): pass
D.__mro__   # D, B, C, A, object
```

Q132. MRO kia hai?

MRO (Method Resolution Order) define karta hai ke Python pehle kis class ke method ko call karega jab multiple classes me same method ho.

Q133. What is Composition over Inheritance principle?

Ye principle kehta hai ke jahan tak mumkin ho, code reuse ke liye Inheritance ke bajaye Composition use ki jani chahiye, kyun ke composition zyada flexible aur loosely coupled hoti hai.

Q134. What is method resolution using C3 linearization?

Python MRO C3 linearization algorithm use karta hai jo consistent aur predictable order me classes ko resolve karta hai, especially multiple inheritance me.

Q135. What are SOLID principles (brief)?

Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, Dependency Inversion - ye 5 principles clean aur maintainable OOP code likhne ke liye use hote hain.

Q136. What is operator overloading?

Operator overloading me hum dunder methods (__add__, __sub__ etc.) use karke built-in operators ka custom behavior define karte hain classes ke liye.

Q137. What is class variable vs instance variable conflict?

Agar instance attribute ka naam class attribute se same ho to instance attribute priority leta hai us specific object ke liye.

Q138. What is a mixin class?

Mixin ek chhoti class hoti hai jo specific functionality provide karti hai aur multiple inheritance ke through dusri classes me add ki jati hai, bina independent use ke.

CHAPTER 3

Data Structures & Algorithms (DSA)

67 sawal: structures, sorting, searching aur problem solving

Yahan arrays, linked lists, stacks, queues, trees, graphs, sorting aur searching ke sawal hain. Har cheez ki Big O yaad rakhein, kyun ke interviewer hamesha speed aur memory ke bare me poochta hai.

Q139. What is Data Structure?

Data Structure ek tareeqa hai data ko organize aur store karne ka taake usay efficiently access aur modify kiya ja sake.

Q140. What is an Algorithm?

Algorithm steps ka ek finite set hai jo kisi problem ko solve karne ke liye follow kiya jata hai.

Q141. Time Complexity kya hoti hai?

Time Complexity batati hai ke input size (n) barhne par algorithm ko complete hone ke liye kitne operations perform karne parenge.

Q142. Space Complexity kya hai?

Space Complexity batati hai ke algorithm ko run hone ke liye kitni extra memory chahiye hoti hai input size ke hisaab se.

Q143. Python/Django mein kaun si complexities sabse important hain?

O(1) Constant - Dictionary lookup, List indexing. O(log n) Logarithmic - Binary Search. O(n) Linear - Single loop. O(n log n) - Efficient sorting (Merge/Quick Sort avg). O(n^2) Quadratic - Nested loops.

Q144. Best, Average aur Worst case complexity kya hoti hai?

Best case sabse kam operations wala scenario hota hai. Average case typical scenario hota hai. Worst case sabse zyada operations wala scenario hota hai - algorithms usually worst case ke basis par judge kiye jate hain.

Q145. Big O notation kya hai?

Big O notation ye batati hai ke input size n barhne par algorithm ke time ya memory ki growth rate upper bound me kitni hoti hai (constants ignore karke). Interview me aksar isay worst case ke tor par bataya jata hai.

Misal: O(1) constant, O(log n) binary search, O(n) ek loop, O(n log n) merge sort, O(n^2) nested loops.

Q146. Array kya hai?

Array ek data structure hai jo same data type ki multiple values ko ek hi variable me contiguous memory me store karta hai. Har element ka index 0 se start hota hai.

Q147. Array ki time complexity kya hai (access, search, insert, delete)?

Array ki complexity: Access O(1). Search O(n) (sorted ho to binary search se O(log n)). End par insert/delete O(1): Python list me append() aur pop() amortized O(1) hain (fixed-size array me jagah khatam ho to naya array banana parta hai). Shuru ya beech me insert/delete O(n), kyun ke baqi elements shift karne parte hain.

Q148. List aur Array mein kya difference hai?

List Python ka built-in data structure hai jo different data types store kar sakti hai. Array same data type ke elements store karti hai, isliye memory efficient aur numerical operations ke liye fast hoti hai.

Q149. Python mein array use hoti hai ya list?

Python mein zyada tar list use hoti hai kyun ke wo different data types store kar sakti hai. array module bhi available hai lekin usme sirf same data type ki values store hoti hain.

Q150. What is a Linked List?

Linked List ek linear data structure hai jisme elements (nodes) memory me contiguous nahi hote, har node apne data aur next node ke address (pointer) ko store karta hai.

Q151. Types of Linked List?

Singly Linked List (sirf next pointer), Doubly Linked List (next aur previous pointer), Circular Linked List (last node first node se connected hota hai).

Q152. Array vs Linked List?

Array: elements contiguous memory me hote hain, index se access O(1), lekin beech me insert/delete O(n) (shifting). Linked List: elements memory me bikhre hote hain, access aur search O(n), lekin agar node ka reference pehle se maloom ho (ya head par kaam ho) to insert/delete O(1) hota hai, kyun ke sirf pointers badalte hain.

Nuqta: kisi position tak pohanchne ke liye linked list me O(n) traversal lagta hai, isliye 'linked list me insertion hamesha O(1)' kehna ghalat

hai.

Q153. Linked List ki time complexity kya hai?

Linked List: Access O(n), Search O(n). Head par insert/delete O(1). End par insert O(n) (tail pointer ho to O(1)). Kisi diye gaye node ke baad insert O(1), lekin us node tak pohanchna O(n). Doubly linked list me diya gaya node delete karna O(1) hai.

Q154. What is a Stack?

Stack ek linear data structure hai jo LIFO (Last In First Out) principle follow karta hai. Operations: push (add), pop (remove), peek (top dekhna).

Q155. Stack ke real-world use cases kya hain?

Undo/Redo functionality, Browser back button, Function call stack (recursion), Expression evaluation (balanced parentheses).

Q156. What is a Queue?

Queue ek linear data structure hai jo FIFO (First In First Out) principle follow karta hai. Operations: enqueue (add), dequeue (remove).

Q157. Types of Queue?

Simple Queue, Circular Queue, Priority Queue, Deque (Double-ended Queue).

Q158. Stack vs Queue?

Stack LIFO follow karta hai (last element pehle nikalta hai). Queue FIFO follow karta hai (pehla element pehle nikalta hai).

Q159. Queue ke real-world use cases kya hain?

Task scheduling, Print queue, Handling requests in a web server, BFS traversal.

Q160. What is Recursion (DSA context)?

Recursion me function khud ko call karta hai chhote sub-problems solve karne ke liye. Ismein Base case (rukne ki condition) aur Recursive case hoti hai.

Q161. Recursion vs Iteration?

Recursion function calls use karta hai aur zyada memory (call stack) leta hai lekin code clean hota hai. Iteration loops use karta hai, kam memory leta hai aur usually fast hoti hai.

Q162. What is a Hash Table / Dictionary?

Hash Table ek data structure hai jo key-value pairs store karta hai aur hash function use karke O(1) average time me lookup, insert aur delete karta hai.

Q163. Python me dictionary ki average time complexity kya hai?

Lookup, Insert, Delete: O(1) average case, lekin worst case O(n) (hash collisions ki wajah se, jo rare hota hai).

Q164. What is a hash collision?

Hash collision tab hoti hai jab do different keys ka hash value same ho jaye. Isay chaining ya open addressing techniques se resolve kiya jata hai.

Q165. What is a Tree data structure?

Tree ek hierarchical (non-linear) data structure hai jisme nodes parent-child relationship me connected hote hain, ek root node se start hota hai.

Q166. What is a Binary Tree?

Binary Tree wo tree hai jisme har node ke maximum 2 children (left aur right) hote hain.

Q167. Binary Search Tree (BST) kya hai?

BST ek binary tree hai jisme left child ki value parent se choti aur right child ki value parent se badi hoti hai. Isse search operation O(log n) me hoti hai (balanced tree me).

Q168. Tree traversal ki types kya hain?

Inorder (Left-Root-Right), Preorder (Root-Left-Right), Postorder (Left-Right-Root), aur Level Order (BFS se).

Q169. BST ki time complexity kya hai?

Average case: Search/Insert/Delete O(log n). Worst case (skewed tree): O(n).

Q170. What is a balanced tree (AVL/Red-Black)?

Balanced tree wo BST hoti hai jisme har node ke left aur right subtree ki height ka difference control me rakha jata hai, taake worst case O(n) na ho aur operations O(log n) me rahein.

Q171. What is a Heap?

Heap ek complete binary tree hai jo Heap Property follow karta hai. Min-Heap me parent child se chota hota hai, Max-Heap me parent child se bara hota hai. Priority Queue implement karne ke liye use hota hai.

Q172. What is a Graph?

Graph ek non-linear data structure hai jo nodes (vertices) aur unke beech connections (edges) par mushtamil hota hai.

Q173. Types of Graphs?

Directed Graph (edges ka direction hota hai), Undirected Graph, Weighted Graph (edges ka weight/cost hota hai), Cyclic aur Acyclic Graph.

Q174. Graph representation ke tareeqe kya hain?

Adjacency Matrix (2D array, O(V^2) space) aur Adjacency List (har node ki list of neighbors, O(V+E) space, zyada efficient sparse graphs ke liye).

Q175. BFS (Breadth First Search) kya hai?

BFS graph/tree ko level by level traverse karta hai, Queue data structure use karke. Shortest path (unweighted graph) find karne ke liye use hota hai.

Q176. DFS (Depth First Search) kya hai?

DFS graph/tree ko ek path par jitna deep ja sakta hai jata hai phir backtrack karta hai, Stack (ya recursion) use karke.

Q177. BFS vs DFS?

BFS level by level chalta hai aur Queue use karta hai; unweighted graph me shortest path ke liye best hai. DFS ek raste par jitna gehra ja sake jata hai phir backtrack karta hai, Stack ya recursion use karta hai; cycle detection aur topological sort me kaam aata hai.

Memory: BFS me bari width wale graph/tree par queue bohat bari ho sakti hai, DFS me gehre (deep) graph par stack bari ho sakti hai. Isliye 'DFS hamesha kam memory leta hai' kehna durust nahi, ye graph ki shakal par depend karta hai. Dono ki time complexity O(V + E) hai.

Q178. What is Sorting?

Sorting data ko ek particular order (ascending/descending) me arrange karne ka process hai.

Q179. Bubble Sort kya hai?

Bubble Sort adjacent elements ko compare karke ghalat order me hon to swap karta hai, aur bar bar pass lagata hai. Worst aur average case O(n^2). Agar ek pass me koi swap na ho to ruk jane wale optimized version me best case (pehle se sorted list) O(n) hota hai. Space O(1), stable hai.

Q180. Selection Sort kya hai?

Selection Sort har pass me minimum element dhoond kar usay sahi position par rakhta hai. Time Complexity: O(n^2).

Q181. Insertion Sort kya hai?

Insertion Sort elements ko ek ek karke already sorted part me sahi jagah insert karta hai. Time Complexity: O(n^2), chhoti/nearly-sorted lists ke liye efficient hai.

Q182. Merge Sort kya hai?

Merge Sort Divide and Conquer approach use karta hai - array ko divide karta hai, sort karta hai phir merge karta hai. Time Complexity: O(n log n), stable sorting algorithm hai.

Q183. Quick Sort kya hai?

Quick Sort ek pivot element choose karta hai aur array ko us se choti aur bari values me partition karta hai. Average Time Complexity: O(n log n), Worst case: O(n^2).

Q184. Merge Sort vs Quick Sort?

Merge Sort stable hota hai aur worst case me bhi O(n log n) guarantee karta hai lekin extra space (O(n)) leta hai. Quick Sort in-place hota hai (kam space) lekin worst case O(n^2) ho sakta hai.

Q185. What is a stable sorting algorithm?

Stable sorting algorithm equal elements ka relative order maintain rakhta hai jaisa original array me tha.

Example: Merge Sort, Insertion Sort.

Q186. Linear Search kya hai?

Linear Search array ke har element ko ek ek karke check karta hai. Time Complexity: O(n).

Q187. Binary Search kya hai?

Binary Search sorted array par kaam karta hai, har step me search space ko aadha kar deta hai. Time Complexity: O(log n).

Q188. Binary Search ki precondition kya hai?

Array sorted hona chahiye, kyun ke Binary Search compare karke decide karta hai left ya right half me search continue karni hai.

Q189. What is a Two Pointer technique?

Two Pointer technique me array me do pointers (aksar dono sirron se) chala kar masla hal kiya jata hai, nested loop se bachne ke liye. Sorted array me pair sum dhoondna iski misal hai, jahan O(n^2) ki jagah O(n) lagta hai. Pair-sum wale tareeqe ke liye array ka sorted hona zaroori hai.

```
def pair_sum(a, target):
    l, r = 0, len(a) - 1
    while l < r:
        s = a[l] + a[r]
        if s == target:
            return l, r
        if s < target:
            l += 1
        else:
            r -= 1
```

Q190. What is Sliding Window technique?

Sliding Window technique subarray/substring problems me use hoti hai jahan ek fixed ya variable size window data par slide karti hai taake nested loops se bacha ja sake. Time Complexity O(n) tak reduce hoti hai.

Q191. What is Dynamic Programming (DP)?

DP ek technique hai jisme complex problem ko chhote overlapping sub-problems me tor kar solve kiya jata hai aur unke results ko store (memoize) kiya jata hai taake dobara calculate na karna pare.

Q192. Memoization vs Tabulation?

Memoization Top-Down approach hai (recursion + caching results). Tabulation Bottom-Up approach hai (iterative tareeqe se table fill karna).

Q193. What is the Fibonacci sequence problem in DP context?

Fibonacci ko naive recursion se solve karna O(2^n) leta hai kyun ke same sub-problems dobara calculate hote hain. DP (memoization/tabulation) se ye O(n) me solve hota hai.

Q194. What is Greedy Algorithm?

Greedy Algorithm har step par locally best (optimal) choice select karta hai is umeed me ke overall bhi best result mile.

Example: Coin change (kuch cases me), Activity selection.

Q195. Greedy vs Dynamic Programming?

Greedy har step par us waqt ka behtareen faisla leta hai aur wapas nahi jata. Ye tez hota hai lekin sirf un masalon me optimal hai jinme greedy-choice property ho (jaise activity selection, Huffman, Dijkstra); warna global optimal ki guarantee nahi (kuch coin systems me ghalat hal deta hai).

DP saare sub-problems ko hal karke unke results store karta hai. Ye un masalon par chalta hai jinme optimal substructure aur overlapping sub-problems hon; tab ye optimal hal deta hai, lekin zyada time/space leta hai. Yani 'DP hamesha optimal deta hai' tab sahi hai jab masle me ye do shartein hon.

Q196. What is Divide and Conquer?

Divide and Conquer masle ko chhote sub-problems me baantta hai, unhein independently hal karta hai, phir natayej ko combine karta hai. Misal: Merge Sort, Quick Sort.

Binary Search me har step par masla aadha ho jata hai lekin sirf ek hissa hal karna parta hai (combine nahi), isliye ise aksar 'decrease and conquer' kehte hain.

Q197. What is Backtracking?

Backtracking ek algorithmic technique hai jo step by step solution build karti hai aur agar koi path invalid nikle to wapas (backtrack) chali jati hai.

Example: N-Queens problem, Sudoku solver.

Q198. What is a Palindrome check problem?

Palindrome check karta hai ke string forward aur backward same padhti hai ya nahi. Two-pointer technique se O(n) time me solve hota hai.

Q199. How to find duplicate elements in an array efficiently?

Hash Set/Dictionary use karke elements ko track kiya jata hai. Agar element pehle se set me mojood ho to duplicate hai. Time Complexity: O(n).

Q200. How to reverse a linked list?

Iteratively teen pointers (prev, current, next) use karke har node ka next pointer reverse kiya jata hai. Time Complexity: O(n).

Q201. What is the two-sum problem approach?

Ek Hash Map use karke har element ka complement (target - element) check kiya jata hai. Agar complement map me mile to pair mil gaya. Time Complexity: O(n) (brute force O(n^2) ke muqable).

Q202. What is the difference between Depth and Height of a tree?

Depth kisi specific node ki root se distance hoti hai. Height us node se sabse door leaf tak ki distance hoti hai. Tree ki height root ki height hoti hai.

Q203. What is a Trie data structure?

Trie (Prefix Tree) ek tree-based data structure hai jo strings ko efficiently store aur search karta hai, especially prefix-based searches (autocomplete) ke liye.

Q204. What is the difference between Array and Hash Map in terms of use case?

Array index-based ordered data ke liye best hai (jaise sequential data). Hash Map key-based fast lookup ke liye best hai jab keys unique aur meaningful hon (jaise IDs, names).

Q205. What are common Big-O complexities interviewers expect you to know?

O(1), O(log n), O(n), O(n log n), O(n^2), O(2^n), O(n!) - inka basic idea aur example algorithms yaad hone chahiye.

CHAPTER 4

Django Framework

68 sawal: models, views, ORM, security aur deployment

4

Django wo framework hai jo aap kaam me roz use karenge. Is chapter me architecture, ORM, views, middleware, security aur performance ke sawal hain. ORM wale sawal par khas tawajjo dein, kyun ke har interview me aate hain.

Q206. What is Django?

Django ek high-level Python web framework hai jo fast development aur clean, pragmatic design ke liye famous hai. Ye "batteries-included" approach follow karta hai (authentication, admin panel, ORM built-in).

Q207. Django Project aur App me kya difference hai?

Django Project poori application hoti hai, jabke Django App us application ka ek specific module ya feature hota hai. Ek project ke andar multiple apps ho sakte hain.

Q208. What is MVT in Django?

MVT (Model-View-Template) Django ka architectural pattern hai jo application ko 3 parts me divide karta hai: Model (data), View (logic), Template (UI).

Q209. What is MVC?

MVC (Model-View-Controller) ek software architectural pattern hai jo application ko Model, View aur Controller me divide karta hai.

Q210. Django me MVC nahi MVT kyun hai?

MVC me Controller user ki request ko sambhal kar Model aur View ko jorta hai. Django me naming thori alag hai: Model (data), View (business logic jo request leti aur response banati hai), Template (HTML presentation). MVC ka Controller wala kaam Django ka framework khud URL dispatcher (urls.py) ke zariye karta hai. Isliye Django ke View ko MVC ke Controller jaisa aur Template ko MVC ke View jaisa samjha ja sakta hai.

Q211. Django Request Lifecycle kya hai?

Request ka safar:

1. Browser request bhejta hai, web server (Nginx/Gunicorn) WSGI/ASGI ke zariye Django tak pohanchata hai. 2. Django HttpRequest object banata hai. 3. Middleware (request side, upar se neeche) chalte hain. 4. URL resolver urls.py me match dhoondta hai. 5. Matching View chalti hai aur zarurat par Model/ORM se database se baat karti hai. 6. View HttpResponse (HTML ya JSON) return karti hai. 7. Middleware (response side, neeche se upar) response ko process karte hain. 8. Response browser ko jata hai.

Q212. Middleware kya hai?

Middleware Django ka component hai jo Request aur Response ke darmiyan kaam karta hai aur request ko View tak aur response ko browser tak jane se pehle process karta hai.

Q213. Middleware kitni baar execute hota hai?

Har middleware request aur response dono taraf kaam karta hai: request view tak jate waqt (upar se neeche) aur response wapas aate waqt (neeche se upar). Ye 'sirf 2 baar' ka rigid rule nahi hai; ek middleware request par hi short-circuit karke seedha response bhi return kar sakta hai (jaise authentication fail), tab baqi middleware aur view chalte hi nahi.

```
class TimingMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response
    def __call__(self, request):
        # view se pehle ka code
        response = self.get_response(request)
        # view ke baad ka code
        return response
```

Q214. Django me Middleware kitni types ke hote hain?

Django me middleware ki 'Request Middleware aur Response Middleware' naam ki do alag types nahi hoti. Middleware ek hi class (ya callable) hota hai jo request aur response dono sambhal sakta hai. Zarurat par is me optional hooks bhi hote hain: process_view(), process_exception() aur process_template_response().

Misal ke tor par SessionMiddleware, AuthenticationMiddleware, CsrfViewMiddleware aur SecurityMiddleware sab isi ek design par bane hain.

Q215. Custom Middleware bana sakte hain?

Haan, apni requirement ke mutabiq custom middleware bana kar settings.py ki MIDDLEWARE list me add kiya ja sakta hai.

Q216. Common Built-in Middleware kaha define hote hain?

settings.py ke MIDDLEWARE list me: SecurityMiddleware, SessionMiddleware, CommonMiddleware, CsrfViewMiddleware, AuthenticationMiddleware, MessageMiddleware, XFrameOptionsMiddleware.

Q217. Model kya hai?

Model Django ki ek Python class hoti hai jo database table ko represent karti hai. Model ke har attribute database me ek field (column) banta hai.

Q218. ORM kya hai?

ORM (Object Relational Mapper) Python objects ke through database ke sath kaam karne ka tareeqa hai, jis se SQL likhne ki bajaye Python code se database operations kiye jate hain.

Q219. Migration kya hai?

Migration Django ka database schema ko update karne ka tareeqa hai.

Q220. makemigrations aur migrate me kya difference hai?

makemigrations model changes ke hisaab se migration files create karta hai. migrate un files ko database par apply karta hai.

Q221. Django Relationships kya hain?

Django Relationships models ke darmiyan connection banane ke liye use hoti hain: ForeignKey (Many-to-One), OneToOneField (One-to-One), ManyToManyField (Many-to-Many).

Q222. CASCADE kya hai?

CASCADE ek on_delete behavior hai. Agar parent object delete ho jaye to us se related child objects bhi delete ho jate hain.

Q223. on_delete ke aur options kya hain?

on_delete ke aur options: PROTECT (delete rok deta hai, ProtectedError), RESTRICT (PROTECT jaisa, lekin kuch cascade soortein allow), SET_NULL (field NULL, null=True zaroori), SET_DEFAULT (default value), SET(value) (diya hua value ya callable), DO_NOTHING (kuch nahi karta, database level integrity error aa sakta hai).

Q224. related_name kya hai?

related_name reverse relationship ka naam define karta hai, jis se parent object se related child objects ko access kar sakte hain.

Q225. Meta class kya hai?

Meta class model ki extra settings define karne ke liye use hoti hai, jaise ordering, table name aur permissions.

Q226. QuerySet kya hai?

QuerySet database se retrieve hone wale objects ka collection hota hai. Ye lazy loading follow karta hai, yani jab tak zarurat na ho query execute nahi hoti.

Q227. all() vs filter()?

all() table ke tamam records return karta hai. filter() condition ke hisaab se records return karta hai.

Q228. filter() vs get()?

filter() multiple records (QuerySet) return kar sakta hai. get() sirf ek object return karta hai.

Q229. get() vs filter() (exceptions)?

get() agar object na mile to DoesNotExist exception deta hai, aur agar multiple mil jayein to MultipleObjectsReturned exception deta hai. filter() empty QuerySet return karta hai, exception nahi deta.

Q230. exclude() kya karta hai?

Jo records condition ko match karte hain unhein hata kar baqi records return karta hai.

Q231. first(), last(), exists(), distinct() kya karte hain?

first() pehla object return karta hai. last() aakhri object return karta hai. distinct() duplicate records remove karta hai. exists() check karta hai QuerySet me record mojood hai ya nahi (True/False).

Q232. annotate(), aggregate(), order_by() kya karte hain?

annotate() har object ke sath extra calculated field add karta hai. aggregate() poori QuerySet par calculation karke ek result return karta hai (Sum, Avg, Count). order_by() records ko sort karta hai.

Q233. Function Based View (FBV) kya hai?

FBV ek simple function hoti hai jo request receive karti hai aur response return karti hai.

Q234. Class Based View (CBV) kya hai?

CBV ek class hoti hai jo GET, POST jaise HTTP methods ko alag methods ki form me handle karti hai.

Q235. FBV vs CBV?

FBV simple, easy, choti applications ke liye better hai. CBV reusable, inheritance support karti hai, complex applications ke liye better hai.

Q236. Generic Views kya hain?

Generic Views Django ki built-in CBVs hain jo common operations (List, Create, Update, Delete) ko asaan banati hain: ListView, DetailView, CreateView, UpdateView, DeleteView.

Q237. Django Signals kya hain?

Signals Django ka ek mechanism hai jo decoupled applications ko ek dusre ke events ke baare me notify karta hai.

Q238. Decoupled ka matlab?

Apps directly ek dusre par depend nahi karti, signal ke through communication hoti hai.

Q239. Django mein commonly used signals kaun se hain?

pre_save, post_save, pre_delete, post_delete, m2m_changed, pre_migrate, post_migrate.

Q240. select_related() vs prefetch_related()?

select_related() ForeignKey/OneToOneField ke liye SQL JOIN use karke ek hi query me related data fetch karta hai. prefetch_related() ManyToMany/Reverse ForeignKey ke liye multiple queries chalata hai aur Python me data combine karta hai.

Q241. N+1 Query Problem kya hai?

Jab ek main query chalti hai aur har related object ke liye alag query chalti hai, to usay N+1 Query Problem kehte hain. Is se performance slow hoti hai, select_related()/prefetch_related() se solve hota hai.

Q242. Database Indexing kya hai?

Index database ka data structure hai jo search, filtering aur sorting ko fast banata hai. Zyada use hone wale fields par index lagaya jata hai.

Q243. Gunicorn kya hai?

Gunicorn ek Python WSGI server hai jo Django application ko run karta hai aur requests process karta hai.

Q244. Nginx kya hai?

Nginx ek Reverse Proxy Server hai jo static/media files serve karta hai, security provide karta hai aur load balancing karta hai.

Q245. collectstatic command kya karti hai?

Ye sab apps ki static files ko STATIC_ROOT folder me collect karti hai taake Nginx unhein serve kar sake.

Q246. CSRF kya hai?

CSRF (Cross-Site Request Forgery) ek attack hai jisme attacker authenticated user se fake request send karwata hai. Django CSRF Token se is se protect karta hai.

Q247. XSS kya hai?

XSS (Cross-Site Scripting) me attacker website me malicious JavaScript inject karta hai. Django auto escaping se is se protection deta hai.

Q248. SQL Injection kya hai?

SQL Injection me attacker input ke zariye malicious SQL query chalane ki koshish karta hai. Django ORM parameterized queries use karta hai, isliye ORM se likhi queries safe hain.

Khatra tab hota hai jab raw() ya cursor.execute() me user input ko f-string ya + se query me jod diya jaye. Hamesha parameters alag dein:

```
# GHALAT: f"SELECT * FROM t WHERE id = {uid}"
cursor.execute('SELECT * FROM t WHERE id = %s', [uid])   # SAHI
```

Q249. CORS kya hai?

CORS (Cross-Origin Resource Sharing) browser ka security mechanism hai. Browser by default ek origin (domain/port) ke frontend ko dusre origin ki API se response parhne nahi deta. Server response headers (Access-Control-Allow-Origin) bhej kar batata hai ke kaun se origins allowed hain.

Django me django-cors-headers package use hoti hai: CORS_ALLOWED_ORIGINS = ['http://localhost:3000']. Production me CORS_ALLOW_ALL_ORIGINS = True se bachein.

Dhyan: CORS sirf browsers par laagu hota hai, ye server ki security (authentication) ka badal nahi.

Q250. settings.py kya hota hai?

settings.py Django project ki configuration file hai jisme database, installed apps, middleware, static files aur security settings define hoti hain.

Q251. urls.py kya kaam karta hai?

urls.py URL patterns ko views ke sath map karta hai taake Django ko pata chale kis URL par kaunsi view call karni hai.

Q252. What is a Django template?

Template HTML file hoti hai jisme Django Template Language (DTL) use karke dynamic data display kia jata hai.

Q253. Static files vs Media files?

Static files CSS, JS, images hoti hain jo developer provide karta hai. Media files wo files hoti hain jo users upload karte hain (jaise profile pictures).

Q254. What is Django Admin Panel?

Django admin ek built-in interface hai jisse models ka data easily manage (CRUD) kiya ja sakta hai bina custom UI banaye.

Q255. What is a Form in Django?

Django Form user input ko validate aur clean karne ke liye use hota hai, HTML forms ko Python classes ke through handle karta hai.

Q256. ModelForm kya hai?

ModelForm Model ke basis par automatically form fields generate karta hai, jis se code kam likhna parta hai.

Q257. What is the difference between Session and Cookies?

Cookies client (browser) side par store hoti hain. Session server side par store hoti hai, sirf session ID cookie me client ko di jati hai.

Q258. What is caching in Django?

Caching frequently used data ko temporarily store karta hai taake bar bar database query na chalani pare, is se performance improve hoti hai.

Q259. Types of caching in Django?

Per-view caching, Template fragment caching, Low-level cache API, aur Whole-site caching.

Q260. What is a context processor?

Context processor ek function hai jo templates ko extra context data provide karta hai jo har request ke sath available hota hai, bina har view me manually pass kiye.

Q261. What is Django's ORM lazy evaluation?

QuerySet database se data tab tak fetch nahi karta jab tak usay actually evaluate na kiya jaye (jaise loop, list(), print). Isay lazy evaluation kehte hain.

Q262. bulk_create() kya hai?

bulk_create() bohat saare objects ko ek hi query me insert karta hai, is liye hazaron objects ke liye individual save() se bohat tez hai.

Dhyan: bulk_create me model ka save() method call nahi hota aur pre_save/post_save signals nahi chalte. Kuch databases me objects ki primary key wapas nahi milti. Isi tarah QuerySet.update() bhi save() aur signals nahi chalata.

```
Product.objects.bulk_create([
    Product(name='A'), Product(name='B'),
])
```

Q263. What is a custom manager in Django models?

Custom Manager ek class hoti hai jo Model.objects ke through custom query methods provide karti hai.

Q264. What is F() expression in Django?

F() expression database field ki value ko dusri field ke sath compare ya update karne ke liye use hota hai, bina Python me value load kiye.

Q265. What is Q() object in Django?

Q() object complex queries banane ke liye use hota hai jisme AND, OR, NOT conditions combine ki ja sakti hain.

Q266. What is the difference between values() and values_list()?

values() dictionaries ka QuerySet return karta hai. values_list() tuples ka QuerySet return karta hai.

Q267. What is Django's transaction.atomic()?

transaction.atomic() ek block of database operations ko atomic banata hai, yani agar koi operation fail ho to sab operations rollback ho jate hain.

Q268. What is a slug field in Django?

Slug field URL-friendly short label hota hai jo usually title se generate hota hai (jaise 'my-first-post').

Q269. What is the difference between abstract base class and multi-table inheritance in models?

Abstract base class sirf common fields provide karti hai, apni table nahi banati. Multi-table inheritance har model ke liye alag table banati hai aur OneToOne link se connect karti hai.

Q270. What is Django Channels?

Django Channels Django ko WebSockets aur asynchronous protocols support karne ki ability deta hai, real-time features (chat, notifications) ke liye.

Q271. What is Celery used for with Django?

Celery ek task queue hai jo Django me background/asynchronous tasks (emails, notifications, heavy processing) run karne ke liye use hoti hai, usually Redis ya RabbitMQ ke sath.

Q272. What is the difference between get_object_or_404 and get()?

get_object_or_404() object na milne par 404 error return karta hai. get() DoesNotExist exception raise karta hai jo manually handle karna parta hai.

Q273. What is Django's static vs staticfiles app?

'static' template tag static files ko URL me reference karta hai. 'staticfiles' app development aur production me static files ko manage/collect karne me madad karta hai.

CHAPTER 5

Django REST Framework (DRF)

47 sawal: serializers, viewsets, JWT aur permissions

5

DRF Django ko ek taqatwar API builder bana deta hai. Yahan serializers, views, viewsets, authentication, permissions, throttling aur pagination seekhein. Backend interviews me yahi topics sabse zyada puchhe jate hain.

Q274. What is Django REST Framework?

DRF Django ka framework hai jo RESTful APIs banane ke liye use hota hai. Isme serializers, authentication, permissions aur pagination jaise features built-in hote hain.

Q275. Serializer kya hota hai?

Serializer Python objects ko JSON me convert karta hai aur JSON data ko Python objects me convert karta hai.

Q276. ModelSerializer kya hai?

ModelSerializer automatically model ke basis par serializer aur fields generate karta hai. Is se code kam likhna parta hai.

Q277. Serializer vs ModelSerializer?

Serializer: fields manually define karni parti hain. ModelSerializer: fields model se automatically generate hoti hain.

Q278. APIView kya hai?

APIView DRF ki base class hai jo APIs banane ke liye use hoti hai aur GET, POST, PUT, DELETE requests handle karti hai.

Q279. GenericAPIView kya hai?

GenericAPIView APIView ka extended version hai jo serializer, queryset, pagination aur filtering ki support deta hai.

Q280. Mixins kya hain?

Mixins reusable classes hoti hain jo common functionality provide karti hain aur multiple views me reuse ki ja sakti hain: CreateModelMixin, ListModelMixin, UpdateModelMixin, DestroyModelMixin.

Q281. ViewSet kya hai?

ViewSet CRUD operations (Create, Read, Update, Delete) ko ek hi class me provide karta hai.

Q282. Router kya hai?

Router ViewSet ke URLs automatically generate karta hai.

Q283. APIView vs ViewSet?

APIView zyada custom control deta hai. ViewSet CRUD APIs ke liye kam code likhna parta hai aur Router ke sath use hota hai.

Q284. APIView ya ViewSet, CRUD ke liye kaunsa better hai?

ViewSet, kyun ke ye CRUD operations ko concise tareeqe se handle karta hai aur Router ke sath automatically URLs generate karta hai.

Q285. ModelViewSet kya hai?

ModelViewSet ek complete CRUD ViewSet hai jo model ke basis par automatically ye actions deta hai: list, create, retrieve, update, partial_update aur destroy. Sirf queryset aur serializer_class dena kafi hai.

```
class ProductViewSet(viewsets.ModelViewSet):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer
```

Q286. ReadOnlyModelViewSet kya hai?

ReadOnlyModelViewSet sirf list aur retrieve (GET) operations provide karta hai, create/update/delete allow nahi karta.

Q287. JWT kya hai?

JWT (JSON Web Token) stateless token-based authentication hai. Login ke baad server signed token deta hai aur client har request ke Authorization: Bearer <token> header me bhejta hai. Server sirf signature verify karta hai, session store nahi karta.

Dhyan: JWT ka payload encrypted nahi hota, sirf Base64 encoded hota hai, isliye usme password jaisi secret cheezein na rakhein. Signature sirf tampering se bachata hai.

Q288. JWT ko Stateless kyun kehte hain?

Kyun ke server user ki session store nahi karta. User ki information token ke andar hoti hai.

Q289. JWT ke structure me kya hota hai?

JWT teen parts par mushtamil hota hai jo dot (.) se alag hote hain: Header (algorithm aur type), Payload (claims jaise user id, expiry), Signature (header + payload ko secret key se sign kiya hua). Format: xxxxx.yyyyy.zzzzz

Pehle do hisse Base64URL encoded hote hain (koi bhi decode kar sakta hai), is liye hassas data payload me na rakhein.

Q290. Access token vs Refresh token?

Access token short-lived hota hai aur API requests authenticate karta hai. Refresh token long-lived hota hai aur naya access token generate karne ke liye use hota hai.

Q291. Permissions kya hoti hain?

Permissions rules hoti hain jo decide karti hain ke kaun kisi API ya resource ko access kar sakta hai aur kya action perform kar sakta hai.

Q292. Common DRF Permissions kya hain?

AllowAny (kisi ko bhi access, login zaroori nahi), IsAuthenticated (sirf authenticated users), IsAdminUser (sirf admin users).

Q293. What is IsAuthenticatedOrReadOnly?

Ye permission unauthenticated users ko sirf read (GET) access deti hai, lekin write operations (POST, PUT, DELETE) ke liye authentication zaroori hoti hai.

Q294. Custom permission kaise banate hain?

BasePermission class ko inherit karke has_permission() aur has_object_permission() methods override kiye jate hain.

Q295. Throttling kya hai?

Throttling ek mechanism hai jo ek user ya client ki API requests ki speed aur quantity ko limit karta hai, taake server overload na ho aur API ka misuse na ho.

Q296. Agar user allowed limit se zyada requests bheje to?

Server uski additional requests ko temporarily block kar deta hai aur HTTP 429 (Too Many Requests) response return karta hai.

Q297. Throttling kyun use karte hain?

Taake koi user bohat zyada requests na bhej sake, server overload na ho, aur sab users ko fair aur secure access mil sake.

Q298. Types of throttling in DRF?

AnonRateThrottle (anonymous users ke liye), UserRateThrottle (authenticated users ke liye), ScopedRateThrottle (specific views ke liye).

Q299. Pagination kya hai?

Pagination large data ko multiple pages me divide karti hai taake performance aur response time better ho.

Q300. Types of pagination in DRF?

PageNumberPagination, LimitOffsetPagination, CursorPagination.

Q301. Validation kya hai?

Validation data save hone se pehle check karti hai ke data valid hai ya nahi.

Q302. DRF mein kitni types ki validation hoti hain?

3 types: Field-Level Validation (specific field, jaise email format), Object-Level Validation (multiple fields compare, jaise password==confirm password), Validators (built-in/custom, jaise unique email).

Q303. Field-level validation kaise likhte hain?

Serializer me validate_<field_name>() method define karke, jaise validate_email(self, value).

Q304. Object-level validation kaise likhte hain?

Serializer me validate(self, data) method override karke multiple fields ko ek sath check kiya jata hai.

Q305. get() vs filter() (DRF context bhi)?

get() sirf ek object return karta hai (DoesNotExist/MultipleObjectsReturned exception de sakta hai). filter() QuerySet return karta hai (empty ho sakta hai), exception nahi deta.

Q306. What is a Router's DefaultRouter?

DefaultRouter automatically CRUD URLs generate karta hai ViewSets ke liye aur ek API root view bhi provide karta hai jo available endpoints list karta hai.

Q307. What is content negotiation in DRF?

Content negotiation decide karta hai ke response kis format (JSON, XML) me client ko bheja jaye, based on Accept header.

Q308. What are Renderers in DRF?

Renderers decide karte hain ke API response kis format me convert hoga, jaise JSONRenderer, BrowsableAPIRenderer.

Q309. What are Parsers in DRF?

Parsers incoming request data ko parse karte hain (JSON, form data, multipart) taake view usay Python data structures ki tarah use kar sake.

Q310. What is HyperlinkedModelSerializer?

Ye ModelSerializer ki tarah hi hai lekin related objects ko primary key ki bajaye hyperlinks (URLs) ke through represent karta hai.

Q311. What is nested serializer in DRF?

Nested serializer ek serializer ke andar dusra serializer use karta hai taake related model ka data bhi nested JSON ki form me mile.

Q312. SerializerMethodField kya hai?

SerializerMethodField ek read-only field hai jiski value ek custom method (get_<field_name>) se calculate hoti hai.

Q313. What is DRF's browsable API?

Browsable API DRF ka built-in web-based interface hai jisse developer browser me directly API test kar sakta hai bina Postman use kiye.

Q314. What is versioning in DRF?

Versioning API ke different versions ko manage karne ka tareeqa hai, jaise URL versioning, Header versioning, Query parameter versioning.

Q315. What is the difference between authentication classes and permission classes?

Authentication classes decide karti hain user kon hai (identity verify). Permission classes decide karti hain us user ko kya access hai.

Q316. What is TokenAuthentication in DRF?

TokenAuthentication ek simple token-based authentication hai jisme user ko login ke baad ek static token milta hai jo har request ke header me bheja jata hai.

Q317. SessionAuthentication vs TokenAuthentication vs JWT?

SessionAuthentication cookies/session use karta hai (browser based). TokenAuthentication simple static token use karta hai. JWT stateless self-contained token use karta hai jisme expiry bhi hoti hai.

Q318. What is the purpose of to_representation() and to_internal_value() in serializers?

to_representation() Python object ko JSON/output format me convert karta hai. to_internal_value() incoming JSON data ko Python native data types me convert karta hai (validation se pehle).

Q319. What is DRF's ObtainAuthToken view?

Ye built-in view hai jo username/password lekar user ko authentication token return karta hai.

Q320. What is the use of format_suffix_patterns in DRF urls?

Ye URL patterns me .json, .api jaise suffixes add karne deta hai taake client specific format request kar sake.

CHAPTER 6

FastAPI Framework

50 sawal: async APIs, Pydantic, dependencies aur deployment

6

FastAPI Python me API banane ka modern aur tez tareeqa hai. Is chapter me type hints, Pydantic, dependency injection, async code, authentication aur testing hain. AI aur microservice projects me bohat popular hai.

Q321. What is FastAPI?

FastAPI ek modern, high-performance Python web framework hai jo APIs banane ke liye use hota hai. Ye Python type hints par based hai aur Starlette (web parts) aur Pydantic (data validation) par bana hota hai.

Q322. FastAPI itni fast kyun hai?

FastAPI ASGI par chalta hai aur async/await support karta hai, is liye I/O-bound kaam (database, API calls) me ek hi worker bohat saari concurrent requests handle kar sakta hai. Starlette aur Pydantic (jo core me optimized hai) bhi iski speed me madad karte hain.

Dhyan: speed ka fayda zyada tar I/O-bound kaam me hai. CPU-heavy kaam me async khud kuch tez nahi karta. 'Django/Flask se kaafi zyada requests per second' application aur setup par depend karta hai, har haal me nahi.

Q323. FastAPI vs Django vs Flask?

Django full-featured, batteries-included framework hai (ORM, admin panel built-in). Flask lightweight, minimal micro-framework hai. FastAPI modern, async-first, high-performance API framework hai jo automatic validation aur documentation deta hai.

Q324. WSGI vs ASGI?

WSGI synchronous standard hai: ek worker/thread ek waqt me ek hi request par kaam karta hai (concurrency ke liye kai workers ya threads chalane parte hain). Is me WebSockets aur async I/O native support nahi hota. ASGI asynchronous standard hai jo async code, WebSockets aur long-lived connections support karta hai; ek hi event loop par kai requests concurrently chal sakti hain.

Django dono (wsgi.py aur asgi.py) support karta hai, FastAPI sirf ASGI par chalta hai (Uvicorn).

Q325. What is Pydantic?

Pydantic ek Python library hai jo data validation aur settings management ke liye use hoti hai. FastAPI ismein request/response data ko validate aur serialize karne ke liye use karta hai.

Q326. Pydantic BaseModel kya hai?

BaseModel Pydantic ki class hai jise inherit karke hum data schema define karte hain, jisme fields ke type hints diye jate hain aur Pydantic automatically validation karta hai.

Q327. FastAPI me path parameter kya hai?

Path parameter URL ke andar directly define hota hai, jaise /items/{item_id}, aur function parameter ke type hint se automatically validate hota hai.

Q328. FastAPI me query parameter kya hai?

Query parameter URL ke '?' ke baad key=value ki form me aata hai, jaise /items?skip=0&limit=10. Function parameters me default values de kar optional bhi banaya ja sakta hai.

Q329. Path parameter vs Query parameter?

Path parameter URL path ka hissa hota hai aur required hota hai (jaise /users/{id}). Query parameter URL ke end me optional filters ke liye hota hai (jaise ?page=1).

Q330. Request Body kya hai FastAPI me?

Request Body wo data hai jo client POST/PUT request ke sath JSON format me bhejta hai. Pydantic model se define kiya jata hai aur automatically parse/validate hota hai.

Q331. FastAPI me automatic documentation kaise generate hoti hai?

FastAPI OpenAPI standard follow karta hai aur automatically Swagger UI (/docs) aur ReDoc (/redoc) documentation generate karta hai, bina extra code likhe.

Q332. Swagger UI kya hai?

Swagger UI ek interactive web-based interface hai jo API endpoints ko test karne aur unki documentation dekhne ke liye use hota hai, FastAPI me /docs par automatically available hota hai.

Q333. Dependency Injection kya hai FastAPI me?

Dependency Injection ek design pattern hai jisme common logic (jaise database connection, authentication) ko 'Depends()' ke through reusable functions me define karke views me inject kiya jata hai.

Q334. Depends() kya karta hai?

Depends() FastAPI ka function hai jo dependency ko route function me inject karta hai, jaise authentication check, database session, ya common query parameters.

Q335. FastAPI me async def vs def route kab use karte hain?

async def tab use karte hain jab route ke andar I/O-bound operations (DB call, API call) async ho. Agar operations synchronous hain (blocking) to simple def use karte hain, FastAPI usay automatically threadpool me run karta hai.

Q336. What is a Path Operation in FastAPI?

Path Operation wo function hai jo kisi specific HTTP method (GET, POST, PUT, DELETE) aur URL path ke liye decorator (@app.get, @app.post) ke sath define hota hai.

Q337. FastAPI me response_model kya hota hai?

response_model parameter decorator me diya jata hai taake response ka data validate ho aur sirf specified fields hi client ko return hon (extra/sensitive fields exclude ho jayein).

Q338. Status codes FastAPI me kaise set karte hain?

status_code parameter decorator me set kiya jata hai, jaise @app.post('/items', status_code=201), ya HTTPException raise karke error status codes bhi return kiye ja sakte hain.

Q339. HTTPException kya hai?

HTTPException FastAPI ki class hai jo custom error response (status code + detail message) return karne ke liye use hoti hai, jaise raise HTTPException(status_code=404, detail='Not Found').

Q340. FastAPI me Middleware kya hota hai?

Middleware FastAPI me har request/response ko process karne ke liye use hota hai (jaise CORS, logging, authentication) @app.middleware('http') decorator ke sath.

Q341. CORS Middleware FastAPI me kaise add karte hain?

CORSMiddleware ko app.add_middleware() ke through add kiya jata hai jisme allowed origins, methods aur headers specify kiye jate hain.

Q342. FastAPI me background tasks kya hain?

BackgroundTasks class use karke aise tasks (jaise email bhejna, logging) run kiye jate hain jo response return hone ke baad background me execute hote hain, client ko wait nahi karna parta.

Q343. FastAPI me database ke sath kaise kaam karte hain?

FastAPI khud ORM nahi deta, isliye SQLAlchemy ya Tortoise ORM jaise libraries ke sath integrate kiya jata hai, aur database session ko Depends() ke through inject kiya jata hai.

Q344. FastAPI me SQLAlchemy kaise use hoti hai?

SQLAlchemy models define karke, Session ko dependency ke through routes me inject kiya jata hai taake database operations (CRUD) perform ho sakein.

Q345. FastAPI me authentication kaise implement hoti hai?

OAuth2PasswordBearer, JWT tokens, ya API keys use karke authentication implement hoti hai. Security dependencies har protected route par Depends() ke sath lagayi jati hain.

Q346. OAuth2PasswordBearer kya hai?

Ye FastAPI ki built-in security utility hai jo token-based authentication (jaise JWT) implement karne me madad karti hai, login endpoint se token receive karta hai.

Q347. FastAPI me file upload kaise hota hai?

UploadFile aur File() ka use karke files upload kiye jate hain, jaise: async def upload(file: UploadFile = File(...)).

Q348. FastAPI me validation errors kaise handle hote hain?

Agar request data Pydantic model ke sath match na kare to FastAPI automatically 422 Unprocessable Entity response detailed error messages ke sath return karta hai.

Q349. Pydantic me Field() kya karta hai?

Field() extra validation rules aur metadata define karne ke liye use hota hai, jaise minimum/maximum length, default value, description.

Q350. Pydantic Validators kya hain?

Pydantic me custom validation ke liye decorator use hota hai. Pydantic v2 me @field_validator (aur @model_validator) hain; v1 ka @validator purana hai aur v2 me deprecated hai.

```
from pydantic import BaseModel, field_validator
class User(BaseModel):
    email: str
    @field_validator('email')
    @classmethod
    def check_email(cls, v):
        if '@' not in v:
            raise ValueError('email galat hai')
        return v.lower()
```

Q351. FastAPI me nested Pydantic models kya hote hain?

Nested models ek Pydantic model ke andar dusra Pydantic model use karte hain taake complex/related data structures represent ho sakein, jaise Address model Order model ke andar.

Q352. What is Starlette in context of FastAPI?

Starlette ek lightweight ASGI framework hai jispar FastAPI bana hota hai. Ye routing, middleware, aur WebSockets jaisi core web functionalities provide karta hai.

Q353. FastAPI me Uvicorn ka kya role hai?

Uvicorn ek ASGI server hai jo FastAPI application ko run karta hai, jaise: uvicorn main:app --reload.

Q354. FastAPI vs Flask performance ka farq kyun hota hai?

Flask traditional tor par WSGI/synchronous hai: ek worker ek waqt me ek request par kaam karta hai, aur jab wo database ya API ka intezar kare to worker khali baitha rehta hai. Concurrency ke liye kai workers/threads chalane parte hain (Gunicorn). FastAPI async hai: ek worker I/O ke intezar ke dauran dusri requests sambhal leta hai.

Is liye farq zyada tar I/O-bound kaam me nazar aata hai. Flask me 'ek waqt me ek request' poore server ki nahi, ek worker ki baat hai.

Q355. FastAPI me Pydantic Settings kya hai?

BaseSettings class environment variables se app configuration (jaise DB URL, secret keys) automatically load karne ke liye use hoti hai.

Q356. FastAPI router kya hota hai?

APIRouter class use karke routes ko modules me organize kiya jata hai, jinhe baad me main app me include_router() se add kiya jata hai.

Q357. FastAPI me WebSockets kaise implement hote hain?

@app.websocket() decorator use karke real-time bidirectional communication (jaise chat, live notifications) implement ki jati hai.

Q358. FastAPI request validation aur Django validation me difference?

FastAPI me validation Pydantic aur type hints se khud ba khud hoti hai: request aate hi runtime par data check hota hai, aur galat data par 422 error milta hai. Ye compile-time checking nahi hai, Python runtime par chalti hai.

Django me Forms ya DRF Serializers use hote hain jahan fields aur validation ke methods (validate_<field>, validate) aap khud likhte hain.

Q359. FastAPI me exception handlers kaise customize karte hain?

@app.exception_handler() decorator use karke specific exceptions ke liye custom response format define kiya ja sakta hai.

Q360. FastAPI me testing kaise hoti hai?

TestClient (Starlette se) use karke, jo requests library jaisa interface deta hai taake endpoints ko bina server run kiye test kiya ja sake.

Q361. Query() aur Path() functions kya karte hain?

Ye FastAPI ke functions hain jo query/path parameters par extra validation (min_length, max_length, regex, description) add karne ke liye use hote hain.

Q362. What is data serialization in FastAPI?

Serialization Python objects (models) ko JSON me convert karne ka process hai jo response bhejte waqt automatically Pydantic ke through hota hai.

Q363. FastAPI me environment-based configuration kaise manage karte hain?

Pydantic BaseSettings ya python-dotenv ke through .env file se configuration load ki jati hai (dev/staging/production ke liye).

Q364. FastAPI ke real-world use cases kya hain?

Microservices, Machine Learning model serving (AI APIs), real-time applications, high-performance REST/GraphQL APIs.

Q365. FastAPI me GraphQL support hai?

Haan, FastAPI Strawberry ya Ariadne jaisi libraries ke sath integrate ho kar GraphQL APIs bhi support kar sakta hai.

Q366. What is the difference between FastAPI's request and response models?

Request model (Pydantic) incoming data validate karta hai. Response model outgoing data ko shape aur filter karta hai, sirf specified fields client ko return hote hain.

Q367. FastAPI me rate limiting kaise implement karte hain?

Third-party libraries jaise slowapi (FastAPI ke liye) ya custom middleware use karke request rate limit implement ki jati hai.

Q368. FastAPI me caching kaise implement karte hain?

fastapi-cache jaisi libraries ya Redis ke sath manually integrate karke response caching implement ki jati hai.

Q369. What is lifespan/startup and shutdown events in FastAPI?

Startup/shutdown par resources (jaise database connection, ML model) load ya band karne ke liye ab lifespan context manager use hota hai. Purana @app.on_event('startup') / ('shutdown') ab deprecated hai.

```
from contextlib import asynccontextmanager
from fastapi import FastAPI
@asynccontextmanager
async def lifespan(app):
    # startup
    yield
    # shutdown
app = FastAPI(lifespan=lifespan)
```

Q370. FastAPI me pagination kaise implement hoti hai?

Query parameters (skip, limit) manually define karke ya fastapi-pagination jaisi library use karke pagination implement ki jati hai.

CHAPTER 7

Artificial Intelligence & Machine Learning

7

55 sawal: ML ki buniyad se LLMs, RAG aur deployment tak

Aaj kal lagbhag har backend interview me AI ke sawal aate hain. Is chapter me machine learning, neural networks, LLMs, RAG aur AI model ko Django ya FastAPI backend me lagane ka tareeqa hai.

Q371. What is AI (Artificial Intelligence)?

AI wo technology hai jisme machines ko insaano jaisi soch, learning aur decision-making ki ability di jati hai taake wo tasks (jaise recognition, prediction, reasoning) khud perform kar sakein.

Q372. AI, Machine Learning aur Deep Learning me difference?

AI ek broad concept hai jisme machines intelligent behavior dikhati hain. Machine Learning AI ka subset hai jisme machine data se seekh kar predictions karti hai. Deep Learning ML ka subset hai jo neural networks (multiple layers) use karta hai.

Q373. What is Machine Learning?

Machine Learning ek technique hai jisme algorithm data se patterns seekh kar bina explicitly programmed hue predictions ya decisions karta hai.

Q374. Types of Machine Learning?

Supervised Learning (labeled data se), Unsupervised Learning (unlabeled data se patterns dhoondna), Reinforcement Learning (reward/punishment se seekhna).

Q375. Supervised Learning kya hai?

Supervised Learning me model ko labeled data (input + correct output) diya jata hai aur wo input-output relationship seekhta hai.

Example: Spam detection, Price prediction.

Q376. Unsupervised Learning kya hai?

Unsupervised Learning me model ko sirf input data diya jata hai (bina labels ke) aur wo khud patterns/groups dhoondta hai.

Example: Customer segmentation (clustering).

Q377. Reinforcement Learning kya hai?

Reinforcement Learning me agent environment ke sath interact karke reward ya penalty ke basis par behavior seekhta hai.

Example: Game playing AI, Robotics.

Q378. Classification vs Regression?

Classification me output ek category/class hoti hai (jaise spam/not spam). Regression me output ek continuous number hoti hai (jaise house price predict karna).

Q379. What is a training set, validation set and test set?

Training set model ko train karne ke liye use hota hai. Validation set model tuning/hyperparameters check karne ke liye. Test set final unbiased performance evaluate karne ke liye use hota hai.

Q380. What is Overfitting?

Overfitting tab hota hai jab model training data ko itna zyada seekh leta hai (noise samet) ke naye/unseen data par acchi performance nahi deta.

Q381. What is Underfitting?

Underfitting tab hota hai jab model itna simple hota hai ke wo training data ke patterns bhi sahi se nahi seekh pata, is se accuracy training aur testing dono par kam hoti hai.

Q382. Overfitting kaise rokte hain?

Regularization (L1/L2), more training data, cross-validation, dropout (neural networks me), aur model complexity kam karke.

Q383. What is Bias-Variance tradeoff?

Bias model ki simplifying assumptions ki wajah se error hai (underfitting). Variance model ki training data ke changes ke prati sensitivity hai (overfitting). Dono ke beech balance achieve karna zaroori hota hai.

Q384. What is Cross-Validation?

Cross-validation ek technique hai jisme data ko multiple folds me divide kiya jata hai aur model ko har fold par train/test karke performance ka average nikala jata hai (jaise K-Fold Cross Validation).

Q385. What is Feature Engineering?

Feature Engineering raw data se meaningful features (variables) create/select/transform karne ka process hai taake model ki performance behtar ho.

Q386. What is Feature Scaling / Normalization?

Feature Scaling data ki values ko ek specific range (jaise 0-1) me convert karti hai taake sab features equally model ko affect karein. Common methods: Min-Max Scaling, Standardization (Z-score).

Q387. Linear Regression kya hai?

Linear Regression ek supervised algorithm hai jo independent aur dependent variables ke beech linear relationship model karta hai, continuous values predict karne ke liye.

Q388. Logistic Regression kya hai?

Logistic Regression ek classification algorithm hai jo probability calculate karta hai (0-1 ke beech) sigmoid function use karke, usually binary classification ke liye.

Q389. Decision Tree kya hai?

Decision Tree ek tree-like model hai jisme data ko features ke basis par split kiya jata hai taake decisions/predictions ki jaa sakein, easily interpretable hota hai.

Q390. Random Forest kya hai?

Random Forest multiple Decision Trees ka ensemble hai jo unke predictions ko combine (voting/averaging) karke better accuracy aur less overfitting deta hai.

Q391. K-Nearest Neighbors (KNN) kya hai?

KNN ek simple algorithm hai jo naye data point ki classification/value uske 'k' nearest neighbors ke basis par decide karta hai.

Q392. Support Vector Machine (SVM) kya hai?

SVM ek classification algorithm hai jo data points ko classes me separate karne ke liye best possible hyperplane (decision boundary) dhoondta hai.

Q393. K-Means Clustering kya hai?

K-Means ek unsupervised algorithm hai jo data ko 'k' clusters me divide karta hai based on similarity, har cluster ka ek centroid hota hai.

Q394. What is a Neural Network?

Neural Network insaani dimagh ke neurons se inspire ek model hai jisme interconnected layers (input, hidden, output) hote hain jo data se complex patterns seekhte hain.

Q395. What is Deep Learning?

Deep Learning Machine Learning ka subset hai jo multiple hidden layers wale neural networks (deep networks) use karta hai complex tasks (image recognition, NLP) ke liye.

Q396. What is an Activation Function?

Activation Function neural network ke neuron ka output decide karti hai aur non-linearity introduce karti hai.

Examples: ReLU, Sigmoid, Tanh, Softmax.

Q397. What is Backpropagation?

Backpropagation wo algorithm hai jo chain rule ki madad se output ki error ko peeche (output se input ki taraf) bhej kar har weight ka gradient (loss me kitni tabdili) calculate karta hai.

Phir Gradient Descent (ya Adam jaisa optimizer) un gradients ko use karke weights update karta hai. Yani backpropagation gradients nikalta hai, aur gradient descent weights update karta hai.

Q398. What is Gradient Descent?

Gradient Descent ek optimization algorithm hai jo model ke loss function ko minimize karne ke liye parameters (weights) ko gradually update karta hai.

Q399. What is a Loss Function?

Loss Function model ki predictions aur actual values ke beech difference (error) measure karta hai.

Examples: Mean Squared Error (regression), Cross-Entropy Loss (classification).

Q400. Epoch, Batch aur Iteration me difference?

Epoch: poora training dataset ek baar model se guzarna. Batch: data ka chhota hissa jo ek step me process hota hai. Iteration: ek batch par ek update step.

Formula: iterations per epoch = total samples / batch size. Misal: 1000 samples aur batch size 100 ho to 1 epoch me 10 iterations hote hain.

Q401. What is CNN (Convolutional Neural Network)?

CNN ek deep learning architecture hai jo mainly images process karne ke liye use hoti hai, convolution layers se features (edges, shapes) automatically extract karta hai.

Q402. What is RNN (Recurrent Neural Network)?

RNN ek neural network hai jo sequential/time-series data (jaise text, speech) process karne ke liye use hoti hai, kyun ke ye pichli information ko yaad rakhti hai.

Q403. What is NLP (Natural Language Processing)?

NLP AI ka wo field hai jo computers ko insaani zaban (text/speech) samajhne, process karne aur generate karne ki ability deta hai.

Q404. What is Tokenization in NLP?

Tokenization text ko chhote units (words, sub-words, ya characters - tokens) me todne ka process hai taake model usay process kar sake.

Q405. What are Embeddings?

Embeddings words ya sentences ko numerical vectors (numbers ki form) me convert karte hain taake unka semantic meaning (context) machine samajh sake.

Q406. What is a Transformer architecture?

Transformer ek deep learning architecture hai jo 'Attention Mechanism' use karta hai poore sequence ko parallel process karne ke liye, ye modern LLMs (GPT, BERT) ki base hai.

Q407. What is Attention Mechanism?

Attention Mechanism model ko decide karne me madad karta hai ke input sequence ke kaunse hisse par zyada focus karna hai kisi specific output generate karte waqt.

Q408. What is an LLM (Large Language Model)?

LLM ek bara neural network model hai (jaise GPT, Claude) jo bohat zyada text data par train hota hai aur human-like text samajhne aur generate karne me able hota hai.

Q409. What is Generative AI?

Generative AI wo AI systems hain jo naya content (text, images, audio, code) generate karte hain based on patterns jo unhone training data se seekhe hain.

Q410. What is Prompt Engineering?

Prompt Engineering ek technique hai jisme LLM se behtar aur accurate responses lene ke liye input (prompt) ko carefully design kiya jata hai.

Q411. What is Fine-tuning in AI?

Fine-tuning ek pre-trained model ko additional specific dataset par dobara train karne ka process hai taake wo particular task/domain me behtar perform kare.

Q412. What is Transfer Learning?

Transfer Learning me ek pre-trained model (jo already kisi bare dataset par train hua ho) ko naye related task ke liye reuse/adapt kiya jata hai, is se training time aur data ki zarurat kam hoti hai.

Q413. What is RAG (Retrieval-Augmented Generation)?

RAG ek technique hai jisme LLM ko response generate karte waqt external knowledge base/documents se relevant information retrieve karke context ke sath diya jata hai, taake accurate aur up-to-date answers milein.

Q414. What is a Vector Database?

Vector Database data ko embeddings (numerical vectors) ki form me store karta hai aur similarity search (semantic search) ke liye use hota hai, jaise Pinecone, Chroma, FAISS.

Q415. What is Hallucination in LLMs?

Hallucination tab hoti hai jab LLM confidently aisi information generate kar deta hai jo factually galat ya fabricated hoti hai.

Q416. What are common Python libraries used in AI/ML?

NumPy (numerical computing), Pandas (data manipulation), Scikit-learn (ML algorithms), TensorFlow aur PyTorch (deep learning), Matplotlib/Seaborn (visualization).

Q417. What is the difference between AI model inference and training?

Training wo phase hai jisme model data se seekhta hai (weights update hoti hain). Inference wo phase hai jisme trained model naye input par prediction/output deta hai.

Q418. What is Precision, Recall and F1-Score?

Precision batati hai ke positive predictions me se kitni sahi thin. Recall batata hai ke actual positives me se model ne kitne correctly identify kiye. F1-Score Precision aur Recall ka harmonic mean hai.

Q419. What is a Confusion Matrix?

Confusion Matrix ek table hai jo classification model ki performance dikhati hai: True Positive, True Negative, False Positive, False Negative ke through.

Q420. What is Accuracy in ML evaluation?

Accuracy total correct predictions ka percentage hai (correct predictions / total predictions), lekin imbalanced datasets me misleading ho sakti hai.

Q421. What is Data Preprocessing in ML?

Data Preprocessing raw data ko clean, transform aur organize karne ka process hai (missing values handle karna, encoding, scaling) taake model use kar sake.

Q422. What is One-Hot Encoding?

One-Hot Encoding categorical data ko binary vectors (0s aur 1s) me convert karta hai taake ML algorithms usay numerically process kar sakein.

Q423. How can AI models be integrated with a Django/FastAPI backend?

Trained model ko pickle/joblib (ya ONNX) se save karte hain. Backend start hone par model sirf ek baar load karte hain (har request par nahi), phir ek API endpoint (Django view ya FastAPI route) input validate karke model se prediction nikalta aur JSON me return karta hai. Bhari models ko alag microservice ya worker (Celery) me chalate hain.

```
import joblib
model = joblib.load('model.pkl')   # app start par ek baar
@app.post('/predict')
def predict(data: Features):
    return {'result': model.predict([[data.x1, data.x2]]).tolist()}
```

Pickle sirf trusted files ke liye use karein.

Q424. What is Model Deployment in AI?

Model Deployment trained ML model ko production environment me available karna hai taake real users/applications usay use kar sakein, usually API ke through (Flask/FastAPI/Django).

Q425. What is an API key and why is it used with AI services (like OpenAI/Anthropic)?

API key ek secret string hai jo third-party AI service (OpenAI, Anthropic) aap ko pehchanne, usage track karne aur billing ke liye deti hai. Isay code me hardcode na karein aur git me commit na karein; environment variable ya secret manager me rakhein, aur frontend (browser) me expose karne ke bajaye backend se call karein.

```
import os
api_key = os.environ['ANTHROPIC_API_KEY']
```

Durust kiye gaye jawabat

Is edition me un sawalon ke jawab durust ya mukammal kiye gaye jo ghalat, adhoore ya gumrah kun the. Neeche har sawal ka number aur wajah hai. Baqi sawalon ke jawab wese hi rakhe gaye hain.

Q1. dynamically typed shamil kiya.

Q2. 'line by line' wali ghalat fehmi durust ki.

Q3. line by line execution wali baat durust ki, implementation ka farq shamil kiya.

Q4. NoneType aur frozenset shamil kiye.

Q5. 'named memory location' wali C-style definition ghalat thi.

Q6. tuple ke andar mutable object ka nuqta aur hashable shamil kiya.

Q7. hashable aur single-element tuple ka nuqta shamil kiya.

Q9. list saaf kar ke likhi, / aur // ka farq shamil kiya.

Q12. keyword-only aur positional-only arguments shamil kiye.

Q19. map ka iterator hona wazeh kiya.

Q22. Python me 2 nahi, 4 scopes (LEGB) hote hain.

Q23. sirf print ki shart nahi, kisi bhi assignment se local ban jata hai.

Q25. else block shamil kiya aur example di.

Q28. __next__ aur StopIteration shamil kiye.

Q39. event loop ka role aur example shamil kiya.

Q40. GIL release aur Python 3.13 ka nuqta shamil kiya.

Q52. __init__.py ab lazmi nahi, ye batana zaroori tha.

Q62. clear() aur example shamil kiye.

Q66. 'deep equality vs shallow equality' naam ka koi Python concept nahi hai, sawal durust kiya.

Q74. sawal ka title jawab se match nahi karta tha.

Q78. pickle ka security khatra shamil kiya.

Q80. type hints runtime par enforce nahi hote, ye zaroori nuqta missing tha.

Q81. sawal ka title jawab se match nahi karta tha.

Q82. 'is' par depend na karne ki wazahat shamil ki.

Q87. range ko 'generator' kehna ghalat tha.

Q90. 'Error handle nahi hota' wali baat ghalat thi, SyntaxError/MemoryError bhi Exception ki subclasses hain.

Q94. wazahat aur isinstance shamil kiya.

Q99. alternative constructor ki misal shamil ki.

Q103. Python me modifiers strict nahi, ye wazeh kiya.

Q105. 'sirf class me use hota hai' ko convention bataya.

Q106. private bhi mukammal tor par band nahi hota, ye wazeh kiya.

Q116. Pythonic tareeqa (@property) aur example shamil ki.

Q124. Python me method overloading directly nahi hoti, jawab adhoora tha.

Q125. singledispatch aur example shamil kiye.

Q130. super() MRO ki agli class deta hai, sirf parent nahi.

Q131. Python ka hal (MRO) aur example shamil kiya.

Q145. Big O aur worst case ka farq wazeh kiya.

Q147. end par insert amortized O(1) hota hai, fixed array me nahi.

Q152. linked list me insert hamesha O(1) nahi hota.

Q153. tail pointer aur node-reference wali complexities wazeh ki.

Q177. 'DFS kam memory leta hai' wali baat hamesha sahi nahi.

Q179. best case O(n) aur stable hona shamil kiya.

Q189. sorted hone ki shart aur code shamil kiya.

Q195. DP ki shartein (optimal substructure) shamil ki.

Q196. Binary Search ko D&C ka asal example kehna adhoora tha.

Q210. MVC aur MVT ke mapping ko saaf kiya.

Q211. lifecycle ko tafseel se likha (WSGI/ASGI aur middleware ka order).

Q213. short-circuit ka nuqta aur modern middleware ka tareeqa shamil kiya.

Q214. middleware ki '2 types' wali baat ghalat thi.

Q223. RESTRICT aur SET() shamil kiye.

Q248. raw query me khatra aur sahi tareeqa shamil kiya.

Q249. CORS server nahi, browser enforce karta hai; ye wazeh kiya.

Q262. save() aur signals na chalne ka nuqta missing tha.

Q285. partial_update shamil kiya aur example di.

Q287. JWT payload encrypted nahi hota, ye zaroori nuqta shamil kiya.

Q289. encoded vs encrypted ka farq shamil kiya.

Q322. speed ka daawa application par depend karta hai, ye wazeh kiya.

Q324. WSGI 'ek waqt me ek request' per worker hota hai, poori server ke liye nahi.

Q350. v1 @validator deprecated hai, v2 ka sahi example shamil kiya.

Q354. 'Flask ek waqt me ek request' poore server ke liye ghalat tha.

Q358. 'compile-time jaisa' kehna ghalat tha, validation runtime par hoti hai.

Q369. on_event deprecated hai, lifespan naya tareeqa hai.

Q397. backprop weights update nahi karta, gradients nikalta hai.

Q400. formula aur misal shamil ki.

Q423. model ek baar load karna aur pickle ki security shamil ki.

Q425. API key ko mehfooz rakhne ka tareeqa shamil kiya.

Apna agla interview pass karein.

Saaf jawab. Asli examples.

Python, Django, DRF, FastAPI, data structures
aur AI ke asli interview sawal. Ek chapter parhein,
khud ko test karein, aur taiyar ho kar
interview dene jayein.

Ek baar seekhein. Aitmaad se samjhayein.

Muhammad Nouman
Founder & CEO, Xvera Labs